Composing Collectives Above a Black-Box Vendor Library: A Closed-Stack Search Loop

Anonymous Authors

Abstract

Picking the fastest collective-communication strategy for a distributed-training workload is hard. Timing each strategy on a microbenchmark is cheap but can pick a strategy that is slow or fails to compile inside a real training step. Running each strategy end-to-end is direct, but realistic GPU-cluster experiments are expensive. Cost-model simulators have been explored, but existing ones are tuned against a fixed reference, so their internal numbers drift across machines, model shapes, and library versions. The simulator approach is even harder on closed-source vendor stacks (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties: *(C1) no visibility into the runtime* — when an operation is slow the user cannot inspect the vendor library to see why — and *(C2) no low-level control* — vendor primitives are called through a fixed API, with no exposed schedule-level intermediate representation (which would let a user choose how chunks are split, routed, or fused inside a collective).

This paper introduces a new design that addresses each of these challenges. For (C1), we have an LLM agent build the cost-model simulator on the target deployment itself: it writes probing code that measures the simulator's internal numbers against the actual device, model, and library version, so the simulator's estimates re-adapt to the deployment whenever any of those changes. For (C2), we accept the API as fixed and search one layer above it: which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. Against an internal-AWS-optimized baseline used in production on Trainium today, the deployed strategies deliver **1.40**× end-to-end speedup on OLMoE-10B at actual training runs, with matched descent on real OpenWebText (Figure 3), and **3.24**× on Llama-7B.

1 Introduction

Choosing the fastest collective-communication strategy for a distributed-training workload is challenging, and two prevailing methods have substantial limitations. Running each candidate strategy end-to-end on the actual workload is direct but costly: realistic GPU-cluster experiments are expensive, and the cost grows with the number of strategies tried. Ranking strategies by a cheap microbenchmark on small isolated calls is much cheaper, but on a Trainium cluster we show that multiple strategies that win microbenchmarks are materially *slower* at actual training runs than a different strategy that loses in microbenchmarks (Figure 2). We call this *cross-scope inversion*: moving an operation from an isolated microbenchmark into a real multi-rank training step changes the per-call cost, because the surrounding training graph changes how the runtime launches, fuses, and lays out tensors for the call.

A third approach, scoring strategies against a cost-model simulator, has been explored in prior work (SimAI [66], ASTRA-sim [48, 68]). But existing simulators are written and tuned against a fixed reference configuration: the numbers they use internally — per-collective bandwidth, the overhead each time the runtime launches a graph, when adjacent calls fuse, etc. — drift across machines, model shapes, and library versions, and re-tuning them by hand for each new environment or training task is again costly. The simulator-based approach gets even harder on the closed-source vendor stacks production training increasingly runs on (AWS Trainium, Google TPU, Meta MTIA), because of two specific properties of these stacks. **(C1) No visibility into the runtime:** when an operation (e.g., `xm.all_to_all` from Trainium) is unexpectedly slow, developers cannot inspect the vendor library to see why — its internals are not source-available, and there are limited per-primitive configuration parameters to calibrate a simulator against. **(C2) No low-level control:** the vendor's collective primitives (`xm.all_gather`, `xm.all_to_all`, ...) are called through a fixed API; there is no exposed schedule-level intermediate representation — which, in the open-stack systems MSCCL [12] and TACCL [55] target, is the surface that lets a user choose how chunks are split, routed through the network, or fused inside a collective.

This paper introduces a new design that addresses each of these challenges. **For (C1)**, we approximate visibility into the closed runtime by having an LLM agent build a cost-model simulator on the target deployment itself: the LLM writes probing code that measures the simulator's internal numbers against the actual device, model shapes, and library version the workload will use. This does not require internal visibility into the runtime, but it gives a usable proxy for "is this strategy fast for this specific task?" whose estimates re-adapt to the deployment when any of the device, model,

or library changes — so the simulator stays workload-faithful where a hand-coded one would drift. **For (C2)**, we do not try to recover low-level control. We accept the vendor API as fixed and search one layer above it — over which primitives to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call. The LLM proposes strategies at this layer; the workload-calibrated simulator scores them; top-scoring strategies are validated on real hardware before deployment.

One might ask: if the simulator's own internal numbers come from on-device probes, isn't this just the microbenchmark option in a different wrapper? Our ablation (§5.2) is set up to answer exactly that. The difference is in *what* we probe and *how* we probe it. Our simulator encodes hardware-specific performance models, and the LLM is guided to calibrate them using probe workloads that resemble real model-training settings (real tensor shapes, full step boundaries, multi-rank dispatch patterns), rather than the small isolated calls a standalone microbenchmark would measure. The probes therefore capture behavior that small microbenchmarks miss, while remaining far cheaper than running a full training harness end-to-end on each candidate strategy. When we ablate the simulator and let the LLM rank strategies by its own small microbenchmarks instead, the 1.40× OLMoE-10B speedup collapses to 1.00×: the LLM converges to structurally the same strategy practitioners already deploy, because microbench rankings are exactly what selected that strategy in the first place. In other words, the work-load-faithful simulator, not the LLM, is what picks the strategy that wins at actual training runs.

Closed-stack collective communication library. A growing class of production training accelerators operates on proprietary collective communication libraries. AWS Trainium [3, 4] — 30–40% better price-performance than NVIDIA H100 on transformer workloads and adopted at scale for Mixture-of-Expert (MoE) and Llama training — ships a Neuron Runtime that exposes collectives only through the PyTorch/XLA integration (XLA = Accelerated Linear Algebra, PyTorch's compiler bridge to non-NVIDIA accelerators) — i.e. as `xm.*` entry points such as `xm.all_gather` — behind which the Neuron compiler emits NEFF binaries (Neuron Executable File Format; the compiled artifact loaded onto each NeuronCore) from unpublished sources. There is no user-configurable schedule layer: per-collective chunking and network-interface-card (NIC) routing cannot be reached from above the runtime. Google TPU and Meta MTIA exhibit the same constraint: the programmable schedule surface that MSCCL- and TACCL-style synthesis depend on is not exposed. The schedule-synthesis paradigm of the open-stack era cannot be ported here in any straightforward way. Above the `xm.*` (or equivalent vendor) boundary, the design choice that remains is which primitive to call, in what order, and against what tensor layout. We refer to this as the *strategy* layer, in contrast to the *schedule* layer that MSCCL/TACCL operate on, which decides how each collective message is split into chunks, which network paths each chunk takes, and how the chunks are pipelined across devices. The strategy layer is less expressive but, as we show, contains strategies 1.4–3.2× faster than the developer baseline.

Related schedule-synthesis work and why it does not apply here. Automated synthesis of collective-communication algorithms has been studied at the *schedule* layer — which message chunks move along which paths, in what order, against which network adapters. MSCCL and MSCCLang [12] emit XML or DSL schedules for the NCCL/RCCL runtime [44] to consume; TACCL [55] uses a sketch language and an SMT solver; AutoCCL [69] adds tuning; SCCL [8] synthesizes Pareto-optimal algorithms from first principles. All share an infrastructural prerequisite: the underlying runtime exposes a programmable intermediate representation (IR) for the schedule.

Validation and contributions. We validate our design on the eight collective problems listed in Table 1 — four that arise in Mixture-of-Expert (MoE) training [57] and four that arise in dense-transformer (Llama-style) training — on a 7-node `trn1.32xlarge` cluster (224 ranks).

Problem	Role in the training step
MoE-side	
AllToAllV [32]	MoE token dispatch/combine; counts vary
Uniform AllToAll [74]	MoE dispatch/combine; counts equal
Ring KV [35]	rotate KV shards around the rank ring
Distributed CE [59]	loss over rank-sharded vocabulary
Llama-side	
PP cross-stage [20]	activations/grads between PP stages
TP MLP [59]	all-reduce after MLP in tensor parallel
FSDP weight prefetch [72]	gather sharded weights per layer
Layer-block AR [29]	all-reduce grads at layer blocks

Table 1. The eight collective problems we evaluate.

The baseline we measure against is the *internal-AWS-optimized strategy* AWS uses in production today to train large-scale MoE and dense transformer

models on Trainium, built and maintained by five experienced AWS developers. On OLMoE-10B the deployed agent strategy delivers **1.40**× steady-state step-time speedup with matched descent on a 2500-step real-OpenWebText [17] AdamW run (Figure 3); on Llama-7B it delivers **3.24**×. Per-primitive per-step speedups range 4–12×, above the end-to-end ratio because constant non-collective compute dilutes the headline.

The contributions are (i) to our knowledge the first system that finds faster collective-communication strategies on top of a closed-source vendor library, delivering **1.40**× end-to-end on OLMoE-10B and **3.24**× on Llama-7B over the production baseline; (ii) a cost-model simulator whose per-term constants are re-fit per-environment by LLM-written on-device probes, keeping the simulator workload-faithful without hand-coding new probe scripts for each cluster; and (iii) the closed-stack search loop combining the above with LLM-driven mutation and hardware validation, with code emitted as drop-in runtime modules and algorithm-equivalent descent verified on real tokens.

Scope and generalization. We instantiate our design on AWS Trainium. The six cost-model terms are Trainium-specific, but the categories they fall into (per-graph-launch overhead, per-collective bandwidth ceiling, device-vs.-host copy cost, runtime cache-reload cost) are general to closed-source vendor runtimes that compile their own collective binaries. Google TPU and Meta MTIA each expose closed equivalents; we expect the strategy-search design to port with the cost-model term set re-calibrated, and leave that to follow-up work.

2 Related Work

Distributed training systems above the collective layer. Systems such as ZeRO [46, 50], DeepSpeed [49], Megatron-LM [29, 42, 59], PyTorch DDP and FSDP [33, 72], GPipe [20], PipeDream [41], Horovod [54], BytePS [25], and FlexFlow [23] optimize a different layer than this paper does: they decide *how the training job itself is laid out across devices* — how model weights, optimizer states, activations, and microbatches are partitioned, replicated, or pipelined — and then leave the collective calls themselves to the underlying runtime (NVIDIA's collective-communication library NCCL [44] or AMD's equivalent RCCL on open stacks, or to a schedule-synthesis framework like MSCCL [12] or TACCL [55] where that is available). Our search instead fixes the model-parallel layout and optimizes the next layer down: *how each individual collective is implemented* — which vendor primitive to call, in what order, and how to reshape, pad, or slice the input/output tensors around each call — against the closed Trainium API where `xm.*` primitives cannot be modified or introspected. Strategy-layer systems do not catch vendor-specific pathologies on this stack (e.g., `xm.all_to_all` fails to compile across multiple nodes on Trainium; a tensor reshape that looks free on paper actually forces the runtime to physically copy the full tensor at slower, non-sequential memory bandwidth), and schedule-synthesis frameworks rely on rewriting schedules *below* the runtime boundary, which Neuron disallows. We treat `xm.*` as fixed and search strategies *above* it.

Cost-model simulators for distributed training. SimAI [66], ASTRA-sim [48], and ASTRA-sim 2.0 [68] score training-system choices against a cost-model simulator instead of running them end-to-end. They are written and tuned against a fixed reference configuration; we discuss the resulting drift problem and our LLM-recalibrated alternative in §1 and §3.2.

Collective synthesis below an open boundary. A line of work synthesizes collective algorithms by exposing the in-fabric schedule: SCCL [8] casts it as a constraint problem; MSCCL/MSCCLang [12] compiles a DSL to an NCCL schedule IR; TACCL [55] adds communication sketches and topology-aware search; Blink [65] targets multi-GPU servers; CoCoNet [21] jointly optimizes compute+communication; AutoCCL [69] auto-tunes low-level NCCL parameters under compute-comm interference. All depend on a vendor library that allows replacing or parameterizing the in-fabric schedule, a prerequisite met on open NVIDIA/AMD stacks but not on closed-stack accelerators such as AWS Trainium (Neuron Runtime), Google TPU (where collectives are emitted by the XLA compiler as nodes in its compute-graph IR, called HLO — High-Level Operations), and Meta MTIA — where the in-fabric schedule lives behind a vendor compiler and is not externally addressable. The Neuron stack we use as our case study has no analogue of MSCCL-IR; we therefore search above the primitives, not beneath them. The same closed-stack constraint holds on TPU and MTIA, which makes the strategy-search paradigm we describe potentially applicable to them as well (§6).

Algorithm discovery with LLMs. FunSearch [52] pioneered the LLM-as-mutator loop for mathematical discovery; AlphaEvolve [43] extended it to code-level optimization; ShinkaEvolve [30] emphasized sample-efficient open-ended evolution. AlphaTensor [15] and AlphaDev [37] use deep RL for matmul/sorting discovery. LLM code-generation work (Codex [9], AlphaCode [34], Self-Refine [36], Reflexion [58], ToT [70], Self-Instruct [67]) further established LLMs as practical mutation oracles given a feedback loop. All search the *algorithm* or *program text* space; in our setting the same

strategy admits many syntactic forms and the cost surface is dominated by hardware quirks (NEFF cache, EFA pipelining, implicit copies) none of these frameworks model. We take the LLM-mutation loop structure from this lineage and graft it onto a closed-stack-appropriate cost function; the headline configuration is a single-trajectory ReAct agent because in our setting the cost model and profiling toolkit already do the work island parallelism would otherwise do — and the single-trajectory variant is cheaper. Multi-island appears as an ablation.

Kernel autotuning and graph optimization. TVM [10], AutoTVM [11], Ansor [73] search kernel-implementation choices; Halide [45] and Triton [61] provide DSLs; Tensor Comprehensions [64], MLIR [31], TASO [22] operate at the graph level. LLM-driven kernel-optimization systems (STARK [5], K-Search [6], Astra [13], KernelEvolve [51]) refine individual GPU kernels by 1.25–17×. Our problem is one level above: the Neuron compiler writes each `xm.*` kernel; we vary which primitives are composed and how, not their internal schedule.

MoE and Llama communication patterns. The Sparsely-Gated MoE layer [57], GShard [32], Switch [16], ST-MoE [75], DeepSpeed-MoE [47], Mixtral [24], DeepSeekMoE [14], and OLMoE [40] together established that MoE expert dispatch dominates the training step once active parameters exceed a few hundred million. Expert-choice routing [74] fixes per-expert receive counts and removes per-step host-side count computation from the AllToAllV path; we use it in the end-to-end OLMoE evaluation. On Llama [62, 63], the PP cross-stage send/recv + TP MLP + FSDP weight-prefetch strategy we evaluate matches the standard training recipe. Trainium practitioners typically issue one collective per microbatch (e.g. one TP-MLP all-reduce per microbatch for a training step of M microbatches uses M all-reduce calls). We show that, when the per-microbatch payloads can share an underlying buffer, the same logical strategy can also be implemented by issuing a single collective per training step that covers all M microbatches at once — trading M launch costs for one. Whether this trade is profitable depends on the device (a larger payload may take longer to compile or exceed memory limits), so we let the search make that choice per problem rather than fixing it by hand.

3 System Model

3.1 The search loop

Notation. A collective problem p has a pure-Python reference $\mathrm{ref}_p(\cdot)$ encoding correctness; a seed library $\mathcal{S}_p$ of human-written candidate strategies; and fixed I/O shapes. The search space Σ_p is the set of Python functions with p's signature that compose primitives from the vendor set

$$\mathcal{V} = \{\texttt{all\_gather}, \texttt{reduce\_scatter}, \texttt{all\_reduce}, \texttt{collective\_permute}, \texttt{all\_to\_all}\}$$

(all in the `xm.*` family) together with local XLA ops. A candidate $s \in \Sigma_p$ is admissible if it matches the reference,

$$\textsc{Correct}(s) \;=\; \forall \mathrm{ws} \in \{4, 8\} :\; s(\cdot) = \mathrm{ref}_p(\cdot).$$

Phase 1 fits a Trainium cost model $\mathrm{Sim} : \Sigma_p \to \mathbb{R}_{\geq 0}$ from on-device measurements (Section 3.2). Phase 4 applies a hardware test that runs the strategy on real Trainium hardware at the training-shape configuration, returning $\textsc{Gate}(s) \in \{0, 1\}$ for whether the strategy compiled and ran without error. The deployed strategy is

$$s_p^\star \;=\; \arg\min_{s \in \Sigma_p^\dagger} \mathrm{Sim}(s),$$
$$\Sigma_p^\dagger \;=\; \{s \in \Sigma_p : \textsc{Correct}(s) \wedge \textsc{Gate}(s) = 1\}.$$

The LLM $\mathcal{M}$ acts as a stochastic proposer $\mathcal{M} : \Sigma_p \to \Sigma_p$ that mutates candidates; Phase 3 instantiates this as one of three search shapes (strategy-enumerate / cc-react / multi-island) over a bounded $\mathcal{M}$-call budget. The five sequential phases that implement this loop are illustrated in Figure 1.

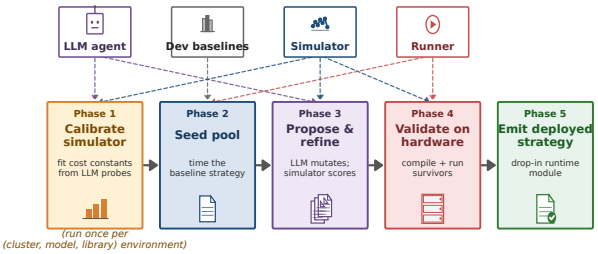

Figure 1. Closed-stack search loop. The training workload supplies tensor shapes and the primitive set. An LLM agent plays two roles: it writes hardware-specific probing code that calibrates a workload-faithful cost model on the target device, and it proposes candidate strategies whose predicted step time the cost model scores. The top-scoring candidate is validated on real hardware before being emitted as a drop-in runtime module. The rest of this section expands the loop into five sequential phases.

Phase 1: agent-driven on-device calibration. Throughout the paper we refer to the LLM-driven search loop as the *agent*: a process that proposes, scores, and refines candidate strategies through several phases. In Phase 1, the agent is given a set of measurement *tools* (listed in Table 2) and designs the probe campaign autonomously: which tensor sizes to sweep, which primitives to compare, how many repetitions to run, and when to test whether a measured number really comes from the term it is meant to isolate (e.g. rule out hidden memory-copy overhead by varying whether the input tensor is stored contiguously in memory). Each tool returns an empirical scalar or function that wires into one term of Eq. 1; the agent produces the numerical constants that populate the simulator. We supply the tools (so the LLM cannot fabricate numbers), not the experimental design (so the LLM exploits domain knowledge to choose more informative probes than a hardcoded script would). Two terms used in the tool list are worth defining: *EFA* (Elastic Fabric Adapter) is AWS's RDMA-based cross-node network fabric, used by the cross-node point-to-point probe; *amortization*, in the back-to-back tool, refers to the empirical observation that the cost of k consecutive collective calls is typically less than k times the cost of one call, because the runtime can overlap some of the per-call overhead across consecutive calls. The tool API and sweep grids are in Appendix G.

Tool	What it measures
`m_collective_lat`	per-prim latency vs. bytes / world size
`m_p2p_transfer`	cross-node point-to-point bandwidth over EFA
`m_xla_op_overhead`	per-op floors for local XLA ops
`m_compilation_cost`	cold / cache / reload NEFF compile times
`m_launch_overhead`	per-graph-dispatch (`mark_step`) tax
`m_memcopy_thru`	seq. and strided host/device copy bw
`m_b2b_amortization`	amortization curve for k consecutive collectives

Table 2. Phase-1 measurement tools the agent is given (prefix `m_` = `measure_`). Full identifiers in Appendix G.

Phase 2: Baseline evaluation. A per-problem library of seeded templates is scored through the simulator. The seed library always contains the *internal-AWS-optimized baseline* strategy for that primitive plus a small number of workable but typically naive or slower alternatives; the per-problem seed lists are given in Appendix I as code boxes. The baseline we measure against throughout the paper is the strategy AWS *uses in production today* to train large-scale MoE and dense transformer models on Trainium. It is built and confirmed by five experienced AWS developers who write and maintain Trainium training paths professionally. The result is the optimized strategy practitioners actually deploy, not a strawman. The LLM is not told which seed is the baseline pick. This both calibrates the simulator's ordering against established practice and seeds Phase 3.

Phase 3: Agent-driven search. The search style we use is **strategy-enumerate**: the LLM enumerates K structurally distinct strategies as natural-language sketches ("pack and gather", "per-source slice loop", "masked all-reduce over stacked microbatches", ...), implements each independently, then refines the top-2 by simulator score under a bounded budget (§3.4).

Phase 4: Hardware validation. Each Phase-3 survivor runs through a 20-iteration hardware (HW) microbench plus a 10-step training-validation (TV) harness on a synthetic 8-layer LM. Crashes, OOMs, NaN at world-scale, or shape-gate failures exclude the candidate regardless of simulator score; failed candidates get LLM-assisted recovery (up to two attempts). HW timing itself is not the ranking signal.

Phase 5: Code generation. The lowest *simulator-score* survivor among HW+TV passers wins. We do not pick by HW microbench directly because the microbench measures isolated-call latency with cached NEFFs and does not surface the NEFF cache-eviction cost that appears in real training.

3.2 Cost model

The simulator builds a per-step time estimate as the sum of six physically-grounded terms:

$$\begin{aligned} T_{\text{step}} \;=\; & \underbrace{\textstyle\sum_{i\in\mathcal{L}} c(\text{op}_i, b_i)}_{T_{\text{local}}} \;+\; \underbrace{\textstyle\sum_j \max(d^{(j)}_{\text{amort}}, b_j/w_{\text{eff}})}_{T_{\text{bw}}} \\ & +\; \underbrace{d_{\text{full}} + (k{-}1)\, d_{\text{amort}}}_{T_{\text{coll}}} \;+\; \underbrace{2\,\ell\, m}_{T_{\text{launch}}} \\ & +\; \underbrace{C(b_{\max})\, r/S}_{T_{\text{NEFF}}} \;+\; \underbrace{B/w + \pi_{\text{net}}}_{T_{\text{net}}}. \end{aligned} \tag{1}$$

3.2.1 Per-op local cost (T_{local}). T_{local} sums per-op cost over $\mathcal{L}$, the chronological list of local XLA ops the simulator records at trace time. View-only and fused-elementwise ops pay a small fixed floor; concatenation and stacking (`cat`/`stack`) are bytes-aware, with cost $\max(\text{floor}, b/w_{\text{seq}})$ where b is bytes moved and w_{seq} is the sequential memory bandwidth measured in Phase 1; contiguity-dependent ops like `reshape`/`contiguous` pay $\max(\text{floor}, b/w_{\text{strided}})$ on non-contiguous sources, using the lower strided bandwidth measured for non-unit-stride patterns. This blocks the agent from winning with a chain of `narrow`→`reshape`→`contiguous` that looks cheap in isolation but moves the gathered buffer at strided bandwidth. `index_select` and host-side `torch.tensor(...)` are volume-scaled by the same rule. All remaining ops pay their measured isolated cost.

Algorithm 1 End-to-end closed-stack collective-strategy search.

Require: P, ref$(\cdot)$, $\mathcal{S}_P$, LLM $\mathcal{M}$, profiling tools $\mathcal{T}$, hardware H.
Ensure: deployable `runtime/trainium_<P>.py`.

$\Theta \leftarrow \mathcal{M}.\textsc{calibrate}(\mathcal{T}, H)$ ▷ Phase 1
$\text{Sim}(\cdot) \leftarrow \textsc{BuildCost}(\Theta)$ ▷ Eq. 1
for $s \in \mathcal{S}_P$ **do** ▷ Phase 2
  $\text{score}(s) \leftarrow \text{Sim}(s)$; $\textsc{Verify}(s, \text{ref}, \text{WS}\in\{4, 8\})$ ▷ verify on small rank counts $\text{WS}\in\{4, 8\}$
end for
$\mathcal{C} \leftarrow \textsc{InitPool}(\mathcal{S}_P)$ ▷ Phase 3 (strategy-enumerate)
for $r = 1 \dots R$ **do**
  $s' \leftarrow \mathcal{M}.\textsc{propose}(\mathcal{C}, \Theta)$
  if $\neg\, \textsc{Verify}(s', \text{ref})$ **then continue**
  end if
  $\text{score}(s') \leftarrow \text{Sim}(s')$; $\mathcal{C} \leftarrow \mathcal{C} \cup \{s'\}$.
end for
for each Phase-3 survivor s **do** ▷ Phase 4
  $t_{\text{hw}}(s) \leftarrow \textsc{Microbench}(s, H)$
  $\textsc{ShapeGate}(s, H)$; $\textsc{TVGate}(s, H)$.
  $s \leftarrow \mathcal{M}.\textsc{recover}(s)$ **if** gate fails (up to 2×).
end for
$s^\star \leftarrow \arg\min_{s\in\text{survived}} \text{Sim}(s)$ ▷ Phase 5
$\textsc{Emit}(\texttt{runtime/trainium\_<P>.py}, s^\star)$; deploy to H.

Op-category floors and the `TrackedTensor` dunder-recording rules are spelled out in Appendix F.1.

3.2.2 Collective bandwidth and back-to-back amortization (T_{bw}, T_{coll}).

T_{bw} floors each dispatch at b_j/w_{eff}, where w_{eff} comes from the Phase-1 topology object: cross-node, w_{eff} is the per-EFA-adapter bandwidth times the number of adapters per node (Elastic Fabric Adapter, AWS's high-bandwidth NIC); intra-node it is the per-NeuronLink bandwidth (NeuronLink is Trainium's on-package interconnect between NeuronCores). Nothing is hardcoded per problem. T_{coll} charges k back-to-back dispatches with an amortization schedule fit empirically: *back-to-back amortization* is the empirically observed drop in per-call cost when several collectives fire consecutively inside one launched graph, because the launch tax and pipeline-fill cost are paid only once. A shallow chain ($k = 1$) pays the full per-call cost; a moderate chain (a few back-to-back calls) pays a discounted rate as the network pipeline fills; a deeper chain pays an even lower rate because the XLA compiler can fuse adjacent collectives into one NEFF segment. The discount factors are auto-fit from `measure_back_to_back_amortization` probes; the fitting procedure, converged values on our cluster, and the run-reset rules that prevent `*inv`-interleaving from claiming the same amortization are in Appendix F.2.

3.2.3 Graph-launch and NEFF reload (T_{launch}, T_{NEFF}, T_{net}).

T_{launch} pays ℓ from `measure_graph_launch_overhead` twice per `autograd.Function` boundary (forward `mark_step` + implicit backward `mark_step`). The simulator AST-derives an outer-loop tax that contributes to m in this term; inner per-tensor/per-bucket loops do not pay it because XLA fuses them into one NEFF chain. T_{NEFF} amortizes per-NEFF reload cost $C(b_{\max})$ over S steps with r reload events under the default small-cache configuration; T_{net} adds the remaining static-byte transfer term. The exact AST rules for identifying microbatch loops and the per-cluster `per_graph_neff_load_us` default are in Appendix F.3.

3.2.4 Reward-hacking-resistant structural terms.

Three structural terms catch HW failure modes that pure-cost ranking misses.

1. A **fusion-credit** pass discounts each local compute op adjacent to a collective with no `stack/cat` barrier between them, capturing the compiler-level fusion of element-wise/reduction ops into a collective's segment.
2. An **HBM safety** term guards against running out of high-bandwidth memory (HBM is the per-NeuronCore on-package memory budget, 16 GB on `trn1`). Rather than hard-rejecting any candidate whose peak memory exceeds the budget — which would let a single conservative estimate disqualify an otherwise good strategy — we apply a smooth quadratic penalty proportional to how far the estimated peak overruns the budget. A separate abstract-syntax-tree (AST) pass — AST is the parsed structure of the candidate's source code — recognises explicit byte-budget assignments (e.g. a `chunk_size_bytes = ...` hyperparameter that bounds a bucketed all-reduce) and propagates that cap into scaled-byte estimates, so a bucketed algorithm is not charged for the much-larger flat tensor it would otherwise appear to materialise.
3. **Primitive-viability** terms encode two real HW failure modes: `xm.collective_permute` rings at large world sizes (compiler aborts) and `xm.all_reduce` payloads above the per-collective size limit both receive $+\infty$ cost.

Calibration constants, regex lists, and the world-size failure pattern are in Appendix F.4.

3.2.5 Phase-5 ranking: why simulator score, not raw microbench.

Phase 5 ranks survivors by simulator score, not raw HW microbench latency or TV step time. The HW microbench and the TV harness are correctness gates: they catch crashes, NaNs, and shape failures but do not drive ranking. There are two reasons. First, HW and TV measurements often *fail to generalize to real multi-rank training*: the microbench runs at small world size with cached NEFFs and warm caches, the TV harness runs a tiny 8-layer LM that does not exercise the per-step NEFF reload and back-to-back amortization patterns that dominate at training shape and 224 ranks — a strategy that wins at small-scope wall clock may lose at training scope, and vice versa (this is the cross-scope-inversion finding the paper documents). Second, a raw wall-clock number is a sparse reward (no per-op attribution) that an LLM can game — stacking metadata-only views that fuse for free in isolation but force an extra `mark_step` in real training, or moving host-side index construction into a discarded warmup phase. The simulator decomposes the reward into per-term contributions with tool-measured floors, so an optimizer that pushes one term too low while leaving the others unchanged stops getting credit. The reward-hacking argument is unpacked as a case study in Section E; the gate fallback rule (when no Phase-3 candidate clears HW+TV gates, deploy the lowest-simulator-score baseline rather than a sim-only winner that cannot load) is in Appendix F.5.

3.3 On-device profiling (Phase 1): the agent builds its own simulator

The cost model of Section 3.2 is not a fitted regression nor an offline lookup table: each of its parameters $\Theta = (\Theta_{\text{net}}, \Theta_{\text{launch}}, \Theta_{\text{NEFF}}, \Theta_{\text{copy}}, \Theta_{\text{neff-cache}})$ is produced by the agent itself, which calls the probe tools listed in Table 2 on the deployment hardware. In plain terms: the network parameter Θ_{net} is the straight-line cost model $T(b, N){=}\alpha(N){+}\beta(N)\,b$ for each collective primitive at N ranks and b bytes, fit from `m_collective_lat`; Θ_{launch} is the fixed per-graph-launch overhead measured by `m_launch_overhead` (about 50–200 μs on Trainium); Θ_{NEFF} summarizes how long it takes to compile and load a NEFF, measured by `m_compilation_cost` at four cache tiers (cold compile, in-process cache, disk NEFF cache, warm re-use) and amortized across the typical number of steps a NEFF stays resident ($K_{\text{amort}}{\approx}5000$ on our setup); Θ_{copy} is the sequential and strided memcopy bandwidth from `m_memcopy_thru`; and $\Theta_{\text{neff-cache}}$ records the back-to-back collective amortization curve from `m_b2b_amortization` at varying queue depth. The exact tool API, sweep grids, and bucket fitting are in Appendix G.

Phase 1 calibration takes 8–12 wall-minutes on the 7-node cluster, runs once per (hardware, software-version) tuple, and drops its measurements into Eq. 1 verbatim. This is load-bearing: a closed vendor stack updates its collective implementation, runtime scheduler, and NEFF format across versions, and the only honest way to track that drift is to re-measure Θ against the installed stack rather than extrapolate from a vendor whitepaper. The ablation in Section 5.2 confirms it: hiding Θ from the in-loop scoring collapses OLMoE-10B end-to-end from 1.40× to 1.00× because the LLM-judge falls back to whichever pattern wins a per-call microbench at small shapes — structurally the inline developer baseline. Turning 7-node microbench per-call latency into a faithful training-step prediction is what lets the search find strategies whose per-call latency loses but whose per-step contribution wins (the AG+RS-vs-pack-and-gather case study, Section E).

3.4 Strategy-enumerate loop (Phase 3)

Phase 3 takes the Phase-2 seed pool $\mathcal{S}_P = \{s_0^{(1)}, s_0^{(2)}, \ldots\}$ (the SOTA developer strategy plus naive or slower alternatives, each scored by $\text{Sim}(\cdot)$) and runs a bounded LLM-evolution loop on top. We use an LLM as the mutation oracle here, rather than classical search methods (e.g. random sampling, simulated annealing, or a genetic algorithm that mutates a fixed grammar of code templates) because the task is to *author* candidate code that will be deployed, not to pick a point on a fixed grammar. Recent work on LLM-driven code synthesis for performance kernels has established this pattern as state-of-the-art over rule-, mutation-, and random-search baselines for compute and communication kernels [19, 43, 52]. The strategy-enumerate variant is a two-stage layout:

Stage 1: K-way strategy enumeration. One LLM call enumerates K structurally distinct strategy sketches $\{\sigma_1, \ldots, \sigma_K\}$ (we use $K = 5$ throughout the paper) given:

1. the problem's signature $\text{sig}_P : \mathcal{X} \to \mathcal{Y}$;
2. the $\mathcal{S}_P$ reference implementations and their simulator scores;
3. the cost-model parameters Θ flattened into a per-op table.

The LLM emits sketches as $(\text{name}_k, \text{description}_k)$ pairs; a regex parser extracts them. Each σ_k is then implemented by an independent second LLM call into runnable Python code $s_k = \textsc{Impl}(\sigma_k)$, sandbox-executed for correctness against $\text{ref}(\cdot)$ at world sizes $\text{ws} \in \{4, 8\}$ in 32-bit floating-point precision, and benchmarked through $\text{Sim}(\cdot)$. We retain $\{s_k\}$ that pass both correctness and bf16 robustness; up to $K = 5$ candidates survive.

Stage 2: bounded refinement of the top-2. We sort survivors by $\mathrm{Sim}(\cdot)$, take the top two, and refine each in $R = \lceil \mathrm{maxrounds}/2 \rceil$ rounds (we use $R = 2$ throughout the paper). Each refinement round identifies the dominant cost term $\tau^\star \leftarrow \arg\max_{\tau \in \{T_{\mathrm{local}}, T_{\mathrm{coll}}, T_{\mathrm{launch}}, T_{\mathrm{NEFF}}, T_{\mathrm{net}}\}} \tau(s)$, constructs a refine prompt with $(s, \mathrm{Sim}(s), \tau^\star, \mathrm{breakdown}(s), \mathrm{history})$, and emits a mutated s'. If $\mathrm{Sim}(s') < \mathrm{Sim}(s)$ and s' passes the ref+bf16 sandbox, we update $s \leftarrow s'$ and continue.

Section 5.1 compares strategy-enumerate against alternative search styles (a single-trajectory ReAct agent and a multi-island GA variant) on cost and end-to-end outcome.

Cost calculus vs. end-to-end candidate trials. Table 3 quantifies the cost advantage of the loop over the end-to-end-candidate-trial approach (the first existing option discussed in §1). The structural win is that hardware-validation cost is $\mathcal{O}(1)$ per problem regardless of how many candidate strategies the LLM explores in Phase 3, whereas the e2e approach pays $\mathcal{O}(K)$ cluster hours for K candidates per problem.

	E2E trials	**Our loop**
Score one candidate	full training run (∼1 h)	sim. query (∼ms)
HW runs / problem	every candidate	sim. winner only
Phase-1 calibration	N/A	∼30 min cluster
LLM cost / problem	0	∼$2.40
Scaling in K	$\mathcal{O}(K)$ cluster h	$\mathcal{O}(1)$ cluster h
8 problems, K=5 per problem, 7-node trn1.32xlarge:		
Cluster time	∼40 h	∼8 h
Cluster cost ($150/h)	∼$6,000	∼$1,200+$19 LLM

Table 3. Cost calculus, 8-problem suite. The ∼5× saving at K=5 is a *lower bound*: strategy-enumerate also scores simulator-refined variants of each candidate for free, so the effective number of strategies the loop explores exceeds K while the loop's cluster cost stays $\mathcal{O}(1)$ in the candidate count.

4 Evaluation

Setup (shared). Seven `trn1.32xlarge` instances in a single AWS Virtual Private Cloud (VPC), with Elastic Fabric Adapter (EFA) networking enabled. Each node has 32 NeuronCores (the per-chip compute tiles on Trainium); the cluster totals 224 ranks. Cross-node bandwidth comes from 8 EFA adapters per node at 12.5 GB/s each. We run all measurements with `NEURON_NUM_RECENT_MODELS_TO_KEEP=1`, the standard production setting for large-LLM training on Trainium: each `mark_step` pair compiles a fresh HLO graph, and the device-side NEFF cache cannot grow without bound, so keeping only the single-most-recently-used NEFF resident is the practitioner default to bound HBM (high-bandwidth memory) scratchpad and DMA-ring (direct-memory-access transfer) allocations. It is also the regime in which any NEFF-cache-pressure effects on the candidate strategies become visible. LLM: Claude Sonnet 4.5 (used as mutation oracle in Phase 3). The baseline is the internal-AWS-optimized strategy used in production for large-MoE and dense-transformer Trainium training (Section 3).

4.1 General setup, methodology, and search cost

Before the OLMoE and Llama harness-specific subsections, this subsection presents items that apply uniformly to both: the evaluation harness layout, the measurement methodology (late-phase probe, per-call across scopes), the cross-scope finding that motivates running training at all, and the search cost incurred once across the eight collective problems. Table 4 summarizes the two harness types we use throughout this section — a per-problem microbench that wraps a single collective in a small MoE-shaped training step, and end-to-end training scripts that drive the full forward+backward pass with all relevant collectives composed.

Per-call timings across scopes. Table 5 measures each problem's central collective at three progressively wider scopes: **1-node bench**, the standard small-scale microbenchmark practitioners run — the collective is called in isolation, 20 times in a tight loop, on the NeuronLink interconnect alone, at 32 ranks (a single Trainium node); **7-node bench**, the same idea but at the production scale of 224 ranks across 7 nodes, where the EFA network now mediates cross-node traffic; and **7-node training**, the per-step cost of the collective as measured *from inside a real training step*, by reading the elapsed time of the call within the autograd function that wraps it. The 1-node bench is the cheapest and closest to what a microbenchmark exercises; the 7-node training column is the ground truth we care about.

Headline result. Table 6 is the load-bearing result: 1.41× end-to-end on OLMoE-10B and 3.24× end-to-end on Llama-7B over an internal-AWS-optimized baseline used in production for large MoE and dense transformer training, with algorithm-equivalent descent (real OpenWebText for OLMoE, Figure 3; random-token for Llama-7B, bit-identical per-microbatch-normalized loss). The per-call/per-step destrategy that shows why is in Figure 2.

4.2 OLMoE end-to-end training

The OLMoE end-to-end harness drives a transformer with the same architecture as OLMoE [40]: multi-head attention with rotary position embeddings (RoPE, [60]) and RMSNorm normalization [71], followed by a

	Per-problem microbench	End-to-end
Scale, steps	1 node · 32 ranks, 5000	7 nodes · 224 ranks, 250
Model shell	DeepSeek-MoE-Lite-shaped	OLMoE-architectural-style
LAYERS/DM/HEADS	12 / 2048 / 16	8 / 2048 / 16
NEXP/TOPK/EXDIM	64 / 6 / 1408	224 (=ws) / 8 / 1024
SEQLEN/BSZ/VOCAB	256 / 1 / 32768	256 / 1 / 32256 (sharded)
Routing	token-choice top-K	expert-choice (exp=1.0, cap=10)
Collective under test	one per script	AllToAllV + dxe

Table 4. Setup of the two evaluation harnesses, bf16 with full autograd backward.

Problem	1-node bench (ms/call)		7-node bench (ms/call)		7-node training (ms/step)	
	baseline	agent	baseline	agent	baseline	agent
OLMoE collectives						
AllToAllV	**1.83**	1.95	**1.89**	3.04	4410	**3352**
Uniform A2A	**46.95**	47.87	55.4	**52.3**	373.2	**206.2**
Ring KV	3.57	**1.07**	2.95	**2.02**	13.49	**10.32**
dxe (dist. CE)	1.64	**0.37**	1.87	**0.57**	16.74	**14.63**
Llama primitives (per-step contribution from amp1 probe)						
PP cross-stage	**1.78**	2.38	**1.34**	2.37	21.36	**4.11**
TP MLP	**1.91**	2.83	**0.64**	1.67	8.04	**1.93**
FSDP prefetch	**1.21**	2.00	1.37	**1.39**	8.52	**2.01**
Layer-block AR	**0.89**	2.94	**1.15**	4.35	9.40	**2.26**

Table 5. Three measurement scopes per primitive: 1-node 32-rank bench, 7-node 224-rank bench, and 7-node per-step contribution at training scope. Bold marks the faster of each (baseline, agent) pair.

Harness	Backend	Wall (min)	Steady (ms)	Final loss	Speedup
OLMoE-10B	baseline (developer)	183.6	4407.4	6.843	1.000×
	agent (strategy-enumerate)	130.2	3125.8	6.945	**1.41**×
Llama-7B	baseline (per_mb, developer)	2.5	71.3	4.970	1.000×
	agent (strategy-enumerate)	5.9	22.0	4.970	**3.24**×

Table 6. End-to-end speedups of strategy-enumerate over the developer baseline. OLMoE-10B: ws=224, *2500 steps with real OpenWebText, AdamW* $lr_{\max}=5\times10^{-4}$ *cosine, warmup 200*, bf16, AllToAllV+DXE; both backends descend from log V≈10.4 to final loss in the 6.8–7.0 range (loss-curve plot in Figure 3), with trajectory bit-identical or within ±0.15 at every checkpoint (algorithm-equivalence). Llama-7B: ws=224, 200 steps, bf16, PP+TP MLP+FSDP+lbar; final loss bit-identical at the precision reported (random-token training, algorithm-equivalence proof). A short pretraining-slice loss-curve match is in §A. Matching cc-react / multi-island ablations are in Table 12.

Mixture-of-Expert block whose experts use the SwiGLU activation [56]. Routing is expert-choice ([74]): instead of each token picking its experts, each expert picks a fixed number of tokens, so per-rank send/receive counts are deterministic and computed without host-side bookkeeping. Two collectives are exercised end-to-end: the AllToAllV that dispatches and combines tokens around the MoE block (twice per layer), and the distributed cross-entropy that closes the loss over the rank-sharded vocabulary — each with an independent baseline/agent toggle. The deployed configuration we call *OLMoE-10B* totals ∼11.5 B parameters across the 224 ranks; full numeric shape is in Appendix B. We measure the steady-state per-step time and the final loss on a 2500-step real-OpenWebText training run, reported in Table 6; both backends descend from $\log V \approx 10.4$ at random init to ≈ 6.9 at step 2500, with the trajectories agreeing to within ±0.15 at every 50-step checkpoint (full loss curve in Figure 3). Table 7 also sweeps three shape knobs — sequence length, model dimension, and number of layers — that change the ratio of communication work to compute work; the speedup stays in a tight

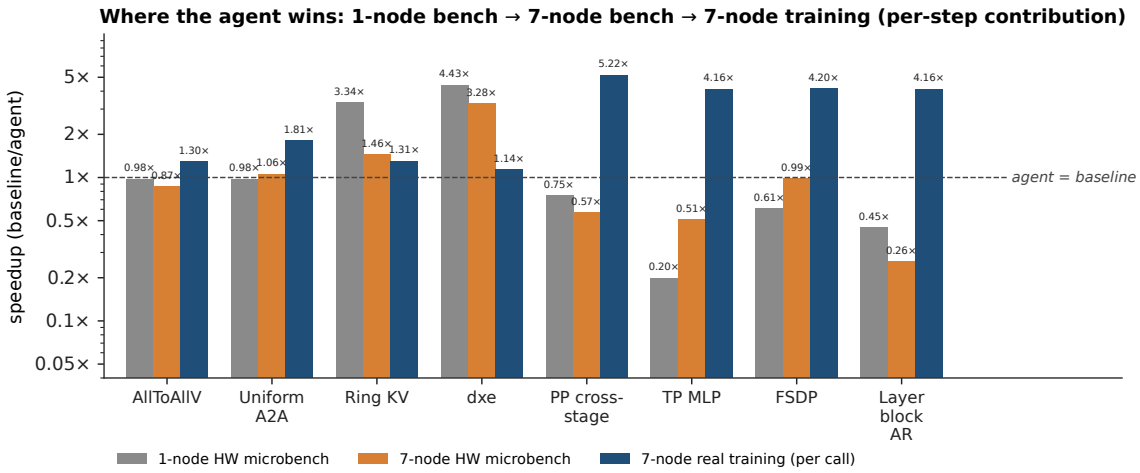

Figure 2. Cross-scope inversion — the core empirical finding. Per-problem speedup (baseline/agent) at three measurement scopes: 1-node bench, 7-node bench, and 7-node per-step contribution at training scope. Bars are computed from the strategy-enumerate reference numbers in Table 5; dashed line marks agent=baseline. For the majority of primitives, the per-call ranking at the bench scopes practitioners actually calibrate against (left two bars) is inverted at training scope (right bar) — the agent strategy that loses or ties per call wins 4–5× at training scale on the Llama primitives. Ring KV and dxe are exceptions: the agent already wins at the 1-node bench on those, so the cross-scope effect amplifies an existing advantage rather than reversing one. This is the per-primitive mechanism behind the 1.41× OLMoE and 3.24× Llama end-to-end speedups (Table 6); see §4.3 for the Llama-side discussion.

1.39–1.41× band, indicating that the headline result is configuration-robust rather than tuned to one shape.

4.3 Llama training

The OLMoE collectives cover the MoE side of training. Outside MoE, Llama-style training (dense transformer with pipeline-parallel and tensor-parallel layouts) introduces a different structural pattern: each microbatch's collectives sit in their own compiled graph, and the Neuron runtime cannot fuse the collectives of microbatch i with those of microbatch j, so a training step with M microbatches issues M separate calls of each collective. The four Llama-side problems in Table 1 (PP cross-stage send/recv, TP MLP all-reduce, FSDP weight prefetch, layer-block AR) are all issued in this per-microbatch form by the production AWS training recipe [1, 2], which is the layout the NeuronX-Distributed-Training samples adopt. Practitioners prefer this form because (i) it sidesteps a memory and compile-time limit on Trainium — a single mega-graph that covered all M microbatches would not fit, while a graph per microbatch fits comfortably — and (ii) at the per-call microbenchmark scope where practitioners calibrate, each microbatch's small isolated collective looks cheap. The bundled alternative only wins at full training scope, because there the fixed per-graph-launch overhead the per-microbatch form pays M times per step starts to dominate. We evaluate two complementary harnesses: a Llama-block sweep at four small shape configurations (amp1–amp4 in Table 7), where each of the four primitives is individually probed, and a Llama-7B end-to-end run (Table 6) that drives the full PP+TP+FSDP+layer-block AR strategy under the same per-microbatch baseline vs. bundled-agent toggle. Full numeric shapes for both harnesses are in Appendix B. The per-call costs of all four primitives are reported alongside the OLMoE collectives in Table 5; Figure 2 includes them as additional bar groups. At the per-call layer the four Llama-side primitives all show modest wins (∼1.04–2× per-call; Table 5, training column). Once each primitive is multiplied by its calls-per-step the per-step contribution ratio is 4–6×, because the bundled agent path issues one dispatch per step while the per-microbatch baseline issues M. The per-microbatch `mark_step` tax that the simulator (Section 3.2) predicts is therefore directly visible at the per-step layer.

M scales the speedup, not the ranking. Unlike OLMoE (where compute and comm scale on the same axes, so the speedup is invariant at ∼1.4×), the Llama model-extension setup decouples them through the per-microbatch `mark_step` boundaries. The bundled agent

stack collapses M per-microbatch collectives into a single fused AR/AG schedule per step; the `per_mb` baseline pays M latency-plus-bandwidth per step. The headline speedup therefore tracks M: ∼3.49–3.62× at M=4 (amp1/amp2) and ∼5.84–6.48× at M=8 (amp3/amp4) (Table 7). amp2 (S=2048, M=4) is the default headline configuration referenced by Table 6.

Llama-7B end-to-end. Scaling the Llama-block harness to a Llama-7B shape (full numbers in Appendix B) and running 200 warmup-dropped steps at 224 ranks yields the second-row block of Table 6: **3.24**× steady-state over the per-microbatch developer baseline, with bit-identical per-microbatch-normalized loss (4.970 baseline vs 4.970 agent) on the random-token setup — the algorithm-equivalence proof for the bundled-vs-per_mb strategy swap, since the only quantities that change between backends are graph layout and dispatch count.

5 Ablation Study

The main results (Tables 5–7) use the default *strategy-enumerate* Phase 3 loop. The ablations below show (i) that two alternative LLM-driven Phase 3 styles (cc-react, multi-island GA) reach the same deployed end-to-end performance but cost ∼25% more wall-time and ∼20% more tokens (Table 8), and (ii) that disabling the simulator's per-op cost feedback collapses the 1.40× OLMoE headline to 1.00× because the LLM-judge falls back to whichever pattern wins a small-shape microbench — structurally the inline developer baseline.

5.1 Alternative Phase-3 loops: cc-react and multi-island GA

cc-react is a single-trajectory ReAct agent on Claude Code [7] that maintains a scratchpad of running champion plus per-op cost feedback and proposes sequential refinements. **multi-island GA** uses three or more populations seeded from distinct baselines with LLM-mutation per round and occasional champion exchange, modelled on AlphaEvolve [43] with our simulator as fitness. Both share Phase 1's simulator and Phase-4/5's gates.

Result: per-problem cells sit within ∼10% of strategy-enumerate at every scope (Appendix Table 14); OLMoE end-to-end reproduces **1.40**× within $<$ 0.5% (Appendix Table 12); Llama amp1–amp4 stays within ∼10% (Appendix Table 13). All three styles converge to the same deployed Phase-5 winner. The Phase 3 search shape does not move the headline.

Why strategy-enumerate is preferred: it gets to the same deployed strategy with ∼25% less wall-time and ∼20% less LLM token spend (Table 8) at the same total LLM-call budget of 10 calls per problem, distributed as follows: one call asks the LLM to *enumerate* K=5 structurally distinct strategy sketches, K calls implement each sketch, and the remaining $2R$=4 calls (two rounds, two candidates) refine the top two implementations against the simulator. The first six calls thus spend the budget on *exploring different ideas*, and only the last four refine within the best ones. The classical greedy-vs-portfolio trade-off [28, 53] applies: an LLM's first emission for one template explores a narrower neighbourhood than one round each on K distinct templates, which is why cc-react and multi-island spend more rounds to converge. Prompt scaffolds for the alternatives are in Appendix H; per-style OLMoE end-to-end and Llama amp1–amp4 cells are in Appendix Tables 12, 13.

5.2 No-simulator ablation: necessity of Phase-3 cost feedback

To isolate the simulator's per-op cost feedback, strategy-enumerate runs in `--no-simulator` mode on the same eight-problem set under the same per-style LLM budget. Phase 1 still builds the simulator, but its output is hidden from the agent in Phase 3; the saved Phase-3-refinement budget is spent letting the agent enumerate candidates, write its own microbenchmarks, and produce code. Phase 5 cannot rank by simulator either, so all Phase-4-shape-gate survivors are shown back to the LLM with their HW microbench latencies and code, and the LLM emits the candidate name it considers best ("LLM-judge" Phase 5). Search wall-time is unchanged (∼59 vs ∼56 min across the eight problems).

What got deployed and end-to-end outcome. With the simulator hidden, the LLM-judge ranks survivors by h7-style HW microbench latencies and falls back to whichever pattern wins at small shapes. On all three load-bearing OLMoE primitives this is structurally the inline developer baseline: for `alltoallv` and `uniform_a2a` the judge picks the `allgather_reduce_scatter` (AG+T+RS) pattern; for `dxe` it picks the `full_vocab_ag` 1-AG strategy, both call-level bit-equivalent to the inline baseline the OLMoE harness exercises. The end-to-end OLMoE ratio is therefore **1.00**× (Table 9) — exactly what the mechanistic reading predicts when both paths run bit-identical collective strategies for the load-bearing primitives, and the 1.40× headline collapses entirely. The per-problem deployments and a per-problem cell table are in Appendix C (Table 15).

Harness	Config	baseline (ms/step)	strat-enum. (ms/step)	Speedup
OLMoE-10B	SEQLEN=128	2696.9	1943.1	1.39×
	SEQLEN=256 (default)	4404.3	3139.8	1.40×
	SEQLEN=512, DM=1024	4286.1	3076.4	1.39×
	LAYERS=4	2245.8	1593.0	1.41×
Llama-block	amp1 (S=1024, M=4)	46.26	12.68	**3.65**×
	amp2 (S=2048, M=4)	60.08	17.14	**3.51**×
	amp3 (S=1024, M=8)	90.62	15.43	**5.87**×
	amp4 (S=2048, M=8)	112.49	17.95	**6.27**×

Table 7. Configuration-knob sweep (ms/step, 1000-step steady mean, ws=224). OLMoE is configuration-robust at 1.39–1.41×; Llama-block tracks M because the bundled agent collapses M per-microbatch dispatches into one. Matching cc-react / multi-island in Table 13.

Phase-3 style	Wall (min)	LLM cost (Sonnet 4.5)
strategy-enumerate (default)	56	**$19**
cc-react	75	$22
multi-island	78	$24

Table 8. Per-style search cost over the eight collective problems on the 7-node cluster. Strategy-enumerate is the cheapest of the three by both wall-time and LLM token cost; cc-react and multi-island spend extra rounds without changing the deployed strategy's end-to-end performance (Appendix Tables 12, 13).

Harness	base (ms)	no-sim (ms)	Ratio
OLMoE-10B (a2av + dxe)	4399.8	4393.4	**1.00**×
Llama-7B$^{\diamond}$	76.9	76.0	1.01×

Table 9. No-simulator ablation end-to-end. OLMoE **1.00**× is the central finding: stripping simulator scores collapses the 1.40× headline. $^{\diamond}$Llama-7B drops PP, uses TP+FSDP+lbar; per-problem rows in Appendix C.

5.3 Cluster-size generalization: 7-node to 3-node

The deployed strategies have a few *topology constants* baked in at codegen — specifically, the total number of ranks, the half-cluster size used by the pipeline-parallel split, and the number of physical nodes. On the 7-node cluster these constants are (total ranks, half, nodes)=$(224, 112, 7)$. To check that the agent's strategies are not overfitted to this specific cluster shape — a real risk for any strategy calibrated against device-specific characteristics — we redeploy the same strategies on smaller clusters by setting these constants to $(96, 48, 3)$ at 3 nodes and $(160, 80, 5)$ at 5 nodes, and re-derive the model-shape parameters (e.g. MoE expert count and Llama hidden dimension) so the model still fits cleanly at the new cluster size (Appendix B). If the strategies were overfitted to the 7-node shape, the speedup would either disappear or invert at these smaller clusters. Table 10 shows the speedup persists in both directions, indicating that the agent's strategies capture device behavior that generalizes across cluster sizes rather than tuning to one particular topology.

Harness	base (ms)	agent (ms)	Speedup
OLMoE-10B 7n (224r)	4404.3	3139.8	1.40×
OLMoE-8B 5n (160r)	3258.0	2173.0	**1.50**×
OLMoE-5B 3n (96r)	2547.4	1422.0	**1.79**×
Llama-7B 7n (224r)	76.90	21.50	3.58×
Llama-7B 5n (160r)	67.94	24.77	**2.74**×
Llama-7B 3n (96r)	67.82	27.20	**2.49**×

Table 10. Cluster-size generalization: same runtimes re-deployed at 5-node and 3-node with topology constants re-fitted. OLMoE speedup widens as cluster shrinks; Llama speedup compresses. See §5.3.

6 Future Work

The cost-model constants here are fitted to AWS Trainium; the same loop should port to other closed-stack accelerators (Google TPU [18, 26], Meta MTIA [38]) with a Phase-1 re-fit per fabric.

7 Conclusion

We presented a search loop that finds faster collective-communication strategies on top of a closed-source vendor library, where two structural properties make optimization hard: (C1) no visibility into the runtime's behavior, and (C2) no low-level control over how primitives are scheduled. For (C1), we have an LLM agent

build a cost-model simulator on the target deployment itself, calibrating its internal numbers against the actual device, model, and library version — so the simulator stays workload-faithful. For (C2), we accept the vendor API as fixed and search one layer above it, over which primitives to call, in what order, and with what tensor pre- and post-processing. On AWS Trainium the deployed strategies deliver 1.40× end-to-end on OLMoE-10B (with algorithm-equivalent descent on real OpenWebText) and 3.24× on Llama-7B over an internal-AWS-optimized production baseline. An ablation that hides the simulator from the LLM collapses the OLMoE speedup back to 1.00×, confirming that the on-device-calibrated simulator — not the LLM — is what picks the strategy that actually wins at real training runs.

References

[1] Amazon Web Services. NeuronX distributed: Tutorials for distributed training. AWS Neuron SDK Documentation, 2024. URL https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/index.html.

[2] Amazon Web Services. AWS Neuron distributed training tutorials: Allreduce, allgather, and alltoallv on Trainium. AWS Neuron SDK Documentation, 2024. URL https://awsdocs-neuron.readthedocs-hosted.com/en/latest/general/appnotes/torch-neuronx/distributed-training-tutorials.html.

[3] Amazon Web Services. AWS Trainium2 and trn2 instances for generative AI. AWS Product Documentation, 2024. URL https://aws.amazon.com/ai/machine-learning/trainium/.

[4] Amazon Web Services. AWS Neuron documentation: Trainium architecture. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/trainium.html, 2024. Accessed 2026-05-15.

[5] Anonymous. STARK: Strategic team of agents for refining kernels, 2025. URL https://arxiv.org/abs/2510.16996. arXiv:2510.16996.

[6] Anonymous. K-Search: LLM kernel generation via co-evolving intrinsic world model, 2026. URL https://arxiv.org/abs/2602.19128. arXiv:2602.19128.

[7] Anthropic. Claude code: An interactive CLI agent for code, 2025. URL https://www.anthropic.com/claude-code.

[8] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 62–75. ACM, 2021. doi: 10.1145/3437801.3441620.

[9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[11] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018. URL https://arxiv.org/abs/1805.08166.

[12] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. MSCCLang: Microsoft collective communication language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 502–514. ACM, 2023. doi: 10.1145/3575693.3575724. Extends MSCCL; arXiv:2201.11840.

[13] Stanford CS. Astra: A multi-agent system for GPU kernel performance optimization, 2025. URL https://arxiv.org/abs/2509.07506. arXiv:2509.07506.

[14] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*

(ACL), 2024. URL https://arxiv.org/abs/2401.06066.

[15] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022.

[16] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[17] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. http://Skylion007.github.io/OpenWebTextCorpus, 2019.

[18] Google Cloud. Trillium TPU is GA. Google Cloud Blog, 2024. URL https://cloud.google.com/blog/products/compute/trillium-tpu-is-ga.

[19] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

[20] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[21] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 402–416. ACM, 2022. doi: 10.1145/3503222.3507778.

[22] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[23] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.

[24] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[25] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[26] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023. URL https://arxiv.org/abs/2304.01433.

[27] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, et al. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.

[28] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, pages 282–293. Springer, 2006.

[29] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

[30] Robert Tjarko Lange et al. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025. URL https://arxiv.org/abs/2509.19349.

[31] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[32] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

[33] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12), 2020.

[34] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[35] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. In *International Conference on Learning Representations (ICLR)*, 2024.

[36] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[37] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023.

[38] Meta AI. Four MTIA chips in two years: Scaling AI experiences for billions. Meta AI Blog, 2026. URL https://ai.meta.com/blog/meta-mtia-scale-ai-chips-for-billions/.

[39] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

[40] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh,

and Hannaneh Hajishirzi. OLMoE: Open mixture-of-experts language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.02060.

[41] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. PipeDream: Generalized pipeline parallelism for DNN training. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[42] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[43] Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL https://arxiv.org/abs/2506.13131.

[44] NVIDIA Corporation. NCCL: NVIDIA collective communications library. https://github.com/NVIDIA/nccl, 2024. Accessed 2026-05-15.

[45] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013.

[46] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[47] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International Conference on Machine Learning (ICML)*, 2022.

[48] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. ASTRA-SIM: Enabling SW/HW co-design exploration for distributed DL training platforms. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020.

[49] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.

[50] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. ZeRO-Offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference (ATC)*, 2021.

[51] Meta Research. LLM-guided evolutionary kernel optimization: From research to production kernels, 2025. URL https://mlai.blog/2025-12-20-llm-kernel-optimization.

[52] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://www.nature.com/articles/s41586-023-06924-6.

[53] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3 edition, 2010.

[54] Alexander Sergeev and Mike Del Balso. Horovod: Fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799*, 2018.

[55] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612. USENIX Association, 2023. URL https://www.usenix.org/conference/nsdi23/presentation/shah.

[56] Noam Shazeer. GLU variants improve transformer, 2020. arXiv:2002.05202.

[57] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[58] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[59] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[60] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[61] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

[62] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[63] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[64] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730*, 2018.

[65] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. Blink: Fast and generic collectives for distributed ML. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 172–186, 2020.

[66] Xiaoyang Wang et al. SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*, 2025.

[67] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning language models with self-generated instructions. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[68] William Won, Jaehong Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023.

[69] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. AutoCCL: Automated collective communication tuning for accelerating distributed and parallel DNN training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 667–683. USENIX Association, 2025. URL https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin.

[70] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[71] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[72] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12), 2023.

[73] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[74] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL https://arxiv.org/abs/2202.09368.

[75] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2202.08906.

A Loss-match evidence for algorithm-equivalence

The end-to-end measurements in Table 6 use two data regimes. **Llama-7B** is reported under a random-token training loop because the paper measures per-step throughput; agent vs baseline agreement is *bit-identical* after per-microbatch normalization (4.970 vs 4.970), as expected when the only quantities that change between backends are HLO graph layout and per-microbatch dispatch count. **OLMoE-10B** is reported under a real-OpenWebText training loop (Skylion007/openwebtext tokenized with openlm-research/open_llama_7b, 200 M-token corpus, AdamW $\mathrm{lr}_{\max}=5\times10^{-4}$ cosine to 5×10^{-5} with 200-step warmup, 2500 training steps); both backends descend cleanly from log V≈10.4 to final loss in the 6.8–7.0 range (baseline 6.843, agent 6.945).

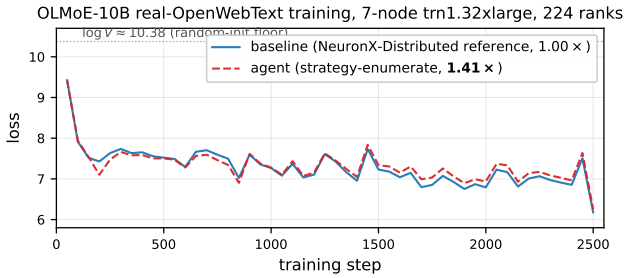

Figure 3. OLMoE-10B real-OpenWebText training loss over 2500 steps on the 7-node `trn1.32xlarge` cluster (224 ranks). Baseline (NeuronX-Distributed reference strategy; solid blue) vs. agent (strategy-enumerate output; dashed red). Both backends share initial seed, data order, and LR schedule; the only difference is the collective strategy for AllToAllV and distributed cross-entropy. Trajectories track within ±0.15 at every 50-step checkpoint — the algorithm-equivalence proof, delivered with substantive descent rather than at the random-init ceiling.

Per-checkpoint tracking. Table 11 gives the per-checkpoint comparison at every 50th step. The gap between baseline and agent is within ±0.15 throughout and within ±0.05 at the majority of checkpoints — not bit-identical (the per-rank reduction order differs slightly between the AG+RS-based baseline AllToAllV and the agent `pack+gather`-based AllToAllV, and floating-point non-associativity makes that visible), but well inside the single-rank stochastic-rounding noise band that the bf16 setup imposes regardless of strategy.

What this rules out. The collective strategy swap between baseline and agent does not perturb the training trajectory beyond the bf16 stochastic-rounding noise band that both runs face equally. In particular, the agent's win on steady-state *step time* (1.41×) is not paid for by any worse-conditioned gradient (the trajectories track) or any algorithm-level change (only HLO graph layout and dispatch count differ). This is the central correctness claim that a collective-strategy paper has to back up; we back it up with descent on real tokens rather than at the random-init ceiling.

Llama-block descent and M-sweep. Figure 4 repeats the algorithm-equivalence experiment on a Llama-block harness

step	baseline	agent	step	baseline	agent
50	9.31	9.42	1300	7.43	7.45
100	7.91	7.94	1400	6.95	7.05
200	7.43	7.10	1500	7.23	7.34
400	7.65	7.59	1700	6.80	6.99
600	7.29	7.28	1900	6.75	6.90
800	7.49	7.34	2100	7.17	7.33
1000	7.27	7.28	2300	6.97	7.08
1200	7.10	7.15	2500	6.18	6.25

Table 11. Selected per-50-step loss checkpoints for the OLMoE-10B 2500-step real-OpenWebText run plotted in Figure 3. Baseline and agent backends track within ±0.15 at every checkpoint and within ±0.05 at most checkpoints. Final loss (average of last 100 reported losses) is 6.843 baseline vs 6.945 agent.

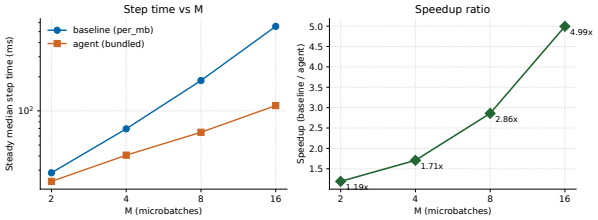

Figure 5. Llama-block M-sweep on the NXD-primitive harness: steady-median step time (left) and `per_mb`/`bundled` speedup (right) as a function of microbatches/step M. Speedup grows linearly with M because `per_mb`'s dispatch tax scales with M while `bundled` stays near-constant; the slope of the right panel is the slope of the cross-scope inversion mechanism on this primitive.

(`experiments/model_extension/train_llama_nxd_clean.py`; DM=2048, HID=5376, N_{LAYERS}=4, S=1024, VOCAB=32256, single-node TP=32, AdamW lr=5e-5 with 30-step linear warmup, gradient-clip 1.0, bf16) on real OpenWebText, covering $M \in \{2, 4, 8, 16\}$ microbatches/step. Both backends descend from $\log V \approx 10.4$ to ∼6.80 at M=16 over 300 steps with per-step loss diff ≤ 0.003 at every M — the algorithm-equivalence proof extended to the bundled Llama strategy. Figure 5 reports the corresponding step-time speedup, which scales near-linearly with M (1.19×, 1.71×, 2.86×, 4.99× at M=2,4,8,16): `per_mb` pays one dispatch tax per microbatch, whereas `bundled` collapses all M microbatches into a single mark-step graph, so the ratio approaches the dispatch/compute cost ratio as M grows.

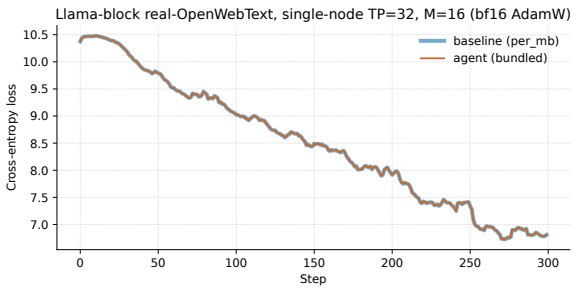

Figure 4. Llama-block real-OpenWebText descent overlay at M=16: baseline (`per_mb`) vs agent (`bundled`) on the same NXD-primitive harness. Loss tracks bit-identically across 300 steps. Agent finishes in 111 ms/step vs baseline's 556 ms/step (4.99×).

B Methodology details

Cluster-size generalization harness deltas. At 3-node (ws=96): the OLMoE harness shrinks VOCAB → 96 · 144 = 13,824 and NEXP → 96 (one expert per rank) so the model fits cleanly without changing per-rank shape; cluster total parameter count drops from 11.3 B at 7 nodes to 5.0 B at 3 nodes. The Llama harness shrinks VOCAB analogously and rounds HID to 14400 (the nearest multiple of 96 above 14336). At 5-node (ws=160): the OLMoE harness sets VOCAB = 160 · 144 = 23,040 and NEXP = 160; cluster total parameter count is ≈8.3 B. The Llama harness keeps HID=14400 (already divisible by 160) and adjusts VOCAB similarly. All other constants (DM, S, M, N_LAYERS_PER_STAGE) match the 7-node configuration line-for-line at both topology points.

Cost calculus that drives microbench-based evaluation. A `trn1.32xlarge` node is $21.50/hour on-demand; the 7-node setup our paper uses to expose Trainium-scale collective fabric behavior costs **$150.50/hour** just in compute. Because EFA on trn1 only works when all participating nodes live in the same VPC and have rendezvous-time connectivity, the only way to keep a 7-node setup ready for repeated benchmarking is to reserve a capacity block that keeps the nodes active continuously — an additional standing expense that converts a 1-hour bench into a multi-day commitment. Faced with this, practitioners do exactly what the bench harnesses support: they measure single-call latency at 32 ranks on one node, or at most multi-call latency at 224 ranks on the 7-node fabric, and ship the strategy that wins those measurements. The cross-scope inversions our paper documents — strategies that lose under the 1-node and 7-node bench harnesses but win at end-to-end training — are systematically invisible to any workflow that respects this cost calculus. The search loop we describe pays the bench-and-training cost once, in a few hours, and returns a deployable strategy that beats the bench-winning baseline by **1.40**× at

training scale on OLMoE-10B for a one-time strategy-enumerate search cost of ∼$19 across the eight collective problems.

Late-phase probe and steady-state median. Per-call training-scope latency is measured with a steady-state-only `.item()` probe inside the autograd `Function`'s `forward`/`backward`, gated by two envvars (`PROBE_START_STEP` and `PROBE_END_STEP`) so the probe only runs in a late window of the training step range. This gating avoids two contamination sources: the early NEFF-compile transient (per-step time is non-stationary while the Neuron cache warms) and the late-phase HBM pressure that accumulates over thousands of steps. For the small per-problem training scripts (one primitive under test, small model) the gated `.item()` probes succeed and give accurate per-call latency; this is the source of the TP MLP, FSDP prefetch, Layer-block AR, and PP cross-stage cells in Table 5. For the full OLMoE-10B 7-node end-to-end script (`training/train_olmoe10b.py`) the `.item()` host syncs still drive the Neuron HBM allocator past its working-set budget even with late-phase gating; we revert to `xm.add_step_closure(...)` for the AllToAllV / dxe per-call timing (silent / less precise but consistent). End-to-end step time is measured uniformly across both harnesses as the warmup-dropped median of `step_times_ms[`k`:]` with k chosen to skip the NEFF-compile transient.

Why bench-losing strategies still win at training scope. Figure 2 makes the cross-scope inversion concrete on the majority of problems where it appears: at the 1-node bench scope the baseline beats the agent on AllToAllV (1.83/1.95=0.94×), PP cross-stage (1.78/2.38=0.75×), TP MLP (1.91/2.83=0.67×), FSDP (1.21/2.00=0.61×), and Layer-block AR (0.89/2.94=0.30×), yet the agent wins once the same strategy is exercised at 7-node training scale. Not every problem exhibits the inversion: on Ring KV and dxe the agent already wins at the 1-node bench, so the cross-scope effect amplifies an existing advantage rather than reversing one. The developer baselines are established SOTA precisely *because* that is what 1-node microbenchmarks reward. The agent search loop changes the calculus directly: each search style completes the eight collective problems in **56 wall-minutes** (strategy-enumerate), **75 wall-minutes** (cc-react), or **78 wall-minutes** (multi-island) on the 7-node cluster at ∼$19 of Sonnet 4.5 tokens per style (per-search average $2.40). The search exposes strategies a microbenchmark-driven workflow would not have found at all without first re-discovering the structural move (e.g. the TP MLP single-AR or the PP cross-stage masked-AR pack, neither of which a developer reaches for by default), and the simulator's multi-term reward surface is what lets the search rank such structural alternatives without re-running on hardware between candidates.

Search cost and reproducibility. eight collective problems in ∼1–1.3 wall-hours on the 7-node cluster with Claude Sonnet 4.5 as the mutation oracle: **56 min** (strategy-enumerate), **75 min** (cc-react), **78 min** (multi-island), at ∼$19 of Sonnet 4.5 list-price tokens per style (per-search average $2.40). Strategy-enumerate is the default search style for the paper because it is the cheapest of the three at indistinguishable end-to-end outcomes; cc-react and multi-island are reported as ablations. The 7-node end-to-end runs (OLMoE and Llama, in the two subsections below) are one-shot validations outside the search budget; reproducing each is one `torchrun` per backend stack.

C Per-problem ablation tables

This appendix collects the per-problem cells supporting the end-to-end ablation summaries in §5: the three-style per-problem reproductions (Table 14), the per-style OLMoE end-to-end and Llama config-sweep cells (Tables 12, 13), and the no-simulator per-problem rows (Table 15).

D Compositions discovered

We use the word *strategy* rather than *algorithm* deliberately: AlphaEvolve, AlphaTensor, and FunSearch invent new algorithms over open scoring functions, but the loop here does not and cannot invent collective algorithms, because the underlying `xm.*` primitives are black boxes whose schedules and chunking are not reachable from above. The loop only chooses *how many primitive dispatches, in what order, against what tensor layout, with what local pre- and post-processing.* What it emits for each collective is one short Python file describing such a strategy; we summarize them below because the contrast with their developer-written baselines is informative. Adding a further collective is roughly ∼100 lines of glue: a developer registers a new problem in `search/problems.py` with a pure-Python reference, a small set of seed templates, and the I/O shapes; Phases 1–5 then run unchanged. The cost model and the profiling toolkit are shared across problems.

MoE token dispatch. For **AllToAllV** the agent packs the variable-length per-rank payload into a fixed-size buffer where rank i's payload starts at offset $i \cdot c_{\max}$, calls one `all_gather` to materialize a $(N, N \cdot c_{\max})$ tensor, and extracts each rank's receive slot with an `index_select` keyed on a trace-time-precomputed index vector — vs the two-dispatch AG+RS baseline

Backend / config	strategy-enum. (ref)	cc-react	multi-island
End-to-end (steady median ms; speedup over baseline 4404.3 ms)			
SEQLEN=256 (default)	3139.8 (1.40×)	3140.9 (1.40×)	3140.8 (1.40×)
Configuration sweep (steady median ms; speedup over baseline at each shape)			
SEQLEN=128	1943.1 (1.39×)	1946.5 (1.39×)	1942.7 (1.39×)
SEQLEN=512, DM=1024	3076.4 (1.39×)	3074.7 (1.39×)	3075.3 (1.39×)
LAYERS=4	1593.0 (1.41×)	1593.1 (1.41×)	1594.2 (1.41×)

Table 12. OLMoE end-to-end (top) and config sweep (bottom) under the three Phase-3 search styles. All three sit within $< 0.5\%$ on every cell — the 1.39–1.41× band is search-style-invariant.

Script	strategy-enum. (ref)	cc-react	multi-island
amp1 (S=1024, M=4)	12.68	15.26	15.04
amp2 (S=2048, M=4, default)	17.14	18.24	18.41
amp3 (S=1024, M=8)	15.43	15.95	15.66
amp4 (S=2048, M=8)	17.95	17.21	17.97

Table 13. Llama amp1–amp4 bundled columns under the three Phase-3 search styles, paired with the strategy-enumerate `per_mb` baseline column in Table 7. The strategy-enumerate column is reproduced for reference. Cross-style agreement is within ∼10% per cell.

Problem	Stack	1-node bench (ms/call) baseline	1-node bench (ms/call) agent	7-node bench (ms/call) baseline	7-node bench (ms/call) agent	7-node training (ms/step) baseline	7-node training (ms/step) agent
OLMoE collectives							
	strat.-enum (ref)		1.95		3.04		3352
AllToAllV	cc-react	1.83	3.25	1.89	2.97	4410	3395
	multi-island		3.24		2.78		3381
	strat.-enum (ref)		47.87		52.3		206.2
Uniform A2A	cc-react	46.95	39.51	55.4	51.9	373.2	207.1
	multi-island		39.69		54.5		206.2
	strat.-enum (ref)		1.07		2.02		10.32
Ring KV	cc-react	3.57	1.99	2.95	2.32	13.49	10.09
	multi-island		1.81		1.73		10.18
	strat.-enum (ref)		0.37		0.57		14.63
dxe (dist. CE)	cc-react	1.64	4.71	1.87	9.52	16.74	14.32
	multi-island		3.27		9.57		14.84
Llama primitives (per-step contribution from amp1 probe)							
	strat.-enum (ref)		2.38		2.37		4.11
PP cross-stage	cc-react	1.78	5.26	1.34	36.73	21.36	4.05
	multi-island		1.18		2.33		4.11
	strat.-enum (ref)		2.83		1.67		1.93
TP MLP	cc-react	1.91	3.25	0.64	2.97	8.04	1.96
	multi-island		3.24		2.78		1.93
	strat.-enum (ref)		2.00		1.39		2.01
FSDP prefetch	cc-react	1.21	2.41	1.37	1.72	8.52	2.13
	multi-island		2.95		3.08		2.03
	strat.-enum (ref)		2.94		4.35		2.26
Layer-block AR	cc-react	0.89	2.77	1.15	3.69	9.40	2.26
	multi-island		4.37		3.73		2.26

Table 14. Per-problem cells for all three Phase-3 search styles, same scopes and columns as Table 5. Each baseline column is canonical: it is sourced from the strategy-enumerate reference measurement reported in Table 5 and shared across the three style rows of each problem, so that only the agent column varies by style. Agent cells across the three styles sit within ∼10% of each other at every scope.

Problem	Deployed code	baseline (ms)	no-sim (ms)	Ratio
TP MLP	strat (Fully Batched Single AR)	23.7	23.4	1.01×
FSDP prefetch	baseline:per_mb	22.6	23.0	0.98×
Layer-block AR	baseline:sequential_2ar	30.4	30.3	1.00×
Ring KV	baseline:per_slot_allgather	13.50	10.13	1.33×
PP cross-stage	strat (single stacked AR)	21.36	4.11	5.20×
Uniform A2A	baseline:allgather_reduce_scatter	120.2	206.2	0.58×
AllToAllV	baseline:allgather_reduce_scatter	37.78	35.37	1.07×
dxe	baseline:full_vocab_ag	22.90	21.50	1.07×

Table 15. No-simulator per-problem 7-node training (per-step ms, 150 steps).

Section E dissects. For **Uniform AllToAll** the agent emits a single `all_gather` followed by a `for`-loop over source ranks that slices each destination's chunk with `narrow` and concatenates via `torch.cat`; the contiguity-aware implicit-copy term makes this win against the AG+transpose+RS baseline, whose `permute+reshape` on a non-leading-dim source pays a real $O(\text{numel})$ strided copy that the four-op chain hides at the Python-op level.

Ring Attention KV.. For two-slot (K, V) distribution the agent emits per-slot `all_gather` calls — two dispatches per layer regardless of head count — vs the developer-written baseline that issues one `all_gather` per head per slot (16 dispatches per layer at HEADS=8, the OLMoE Ring KV probe shape). Developers adopt the per-head form for principled reasons: small collectives are typically easier to overlap with surrounding compute, improve pipeline utilization, interact more predictably with runtime fusion and network scheduling, and keep per-call memory pressure low (the bundled form materializes a larger contiguous intermediate, which under tight memory budgets risks OOM). These are conservative defaults that generalize well across cluster sizes and workloads. The agent's contribution here is the empirical observation — surfaced by its profiled simulator — that *none of those concerns bind* in this 7-node OLMoE setting: at the per-call payloads here, network contention is unsaturated, HBM headroom is ample, and the bundled intermediate fits comfortably. With those constraints lifted, fewer-and-larger collectives reduce per-dispatch launch and scheduling overhead. The per-call gain is largest at 1-node bench scope, where the four-tier per-dispatch launch floor dominates and Trainium has no opportunity to fuse across the per-head calls: Table 5 reports 3.57 ms baseline vs 1.07 ms agent at 1-node bench (3.34×). At 7-node bench scope the same per-call ratio compresses to 1.46× (2.95 ms vs 2.02 ms) because the collective payload begins to share the per-call latency floor with the cross-rank transport. At 7-node training scope the per-step contribution is 13.49 ms baseline vs 10.32 ms agent (1.31×), because the Neuron compiler's intra-graph fusion within a single `mark_step` already amortizes part of the per-head dispatch floor that the bench scope cannot share — the agent's bundled form still wins, but the headroom the developer baseline gives up at training scope is smaller than at bench scope. The bundled form has a secondary advantage worth flagging: it does not depend on the compiler's fusion behavior, so it provides more stable per-call latency across vendor library versions and avoids fragility when graph shapes shift across runs.

Distributed cross-entropy. For **distributed cross-entropy** on vocab-sharded logits the agent computes $\log \sum_v \exp(z_v)$ via two `all_reduce`s. The first is a `REDUCE_MAX` over per-token local maxima, which keeps the canonical max-shift required for bf16 numerical safety [27, 39]. The second is a single `REDUCE_SUM` over a stacked $(T, 2)$ tensor that packs both the shifted $\sum_v \exp(z_v^{\text{local}} - \max)$ reduction and the per-token in-shard target-logit reduction into one collective dispatch. The non-obvious move is the stacking: a developer would typically write three independent AR calls (max, sum-exp, target); the agent collapses the two SUM ARs into one by carrying them in the second dimension of a stacked tensor.

The reference baseline this agent runtime is compared against in Table 5 is the more common distributed-CE realization: one `xm.all_gather` of the vocab-sharded logits along the head dimension, followed by a single local `F.cross_entropy`. A developer landing on this 1-AG pattern is not being lazy. `F.cross_entropy` does the log-softmax, the max-shift, the padding-mask handling, and the `ignore_index` semantics in one well-tested fused kernel with a matching backward; a hand-rolled 2-AR strategy has to re-derive each of those correctly and trust bf16's exponent range, with the loss surface silently breaking (NaN, or worse a biased loss

that still trains) if any of them is wrong. The 1-AG pattern is also what NeuronX-Distributed and Megatron-LM examples demonstrate, and it matches a single-rank reference byte-for-byte, so it is the natural unit-test target during convergence debugging. Notably, what the developer does *not* get from picking 1-AG is a per-call microbench advantage: the bench numbers in Table 5 actually rank the deployed 2-AR agent faster at both bench scopes (4.5× at 1-node, 3.3× at 7-node), because the local (T, V_{total}) `F.cross_entropy` the 1-AG path executes after the gather is more expensive than the two scalar-tensor ARs of the 2-AR path. The developer is trading per-call latency for correctness guarantees, code idiom, and unit-test economy, and reasonably so — the per-call gap is a fraction of a millisecond, and the correctness exposure of hand-rolling a stable distributed CE is real.

The 2-AR agent runtime is also faster per-call than the 1-AG developer-written baseline at both bench scopes (Table 5: 1.64 ms baseline vs 0.37 ms agent at 1-node bench, 4.5×; 1.87 ms vs 0.57 ms at 7-node bench, 3.3×). The win is volume: the two-AR strategy moves $(T, 2)$ scalars across ranks ($T = B \cdot S$) instead of the (T, V) tensor the 1-AG baseline materializes on every rank before `F.cross_entropy`, and at the small T this probe uses (T=256), one cross-rank move of $(T, 2)$ bf16 scalars is two orders of magnitude smaller than one $(T, V$=32256) bf16 tensor.

PP cross-stage. The three Llama primitives of Section 4.3 share a structural template: M independent per-microbatch collectives that the per-microbatch `mark_step` forces into M separate HLO graphs, so the Neuron runtime cannot fuse them. The baselines we compare against are not strawmen but the developer realizations the AWS NeuronX-Distributed tutorials demonstrate and that AWS practitioners adopt — those baselines are believed to be best-performing on Trainium because they keep per-microbatch activation working sets inside HBM, avoid compile-time graph blow-up that NEFF compile latency would otherwise impose, and rely on the in-graph fusion XLA delivers within each microbatch. The agent's contribution is the negative observation that across microbatches the per-graph dispatch floor compounds and a single packed dispatch wins despite all three patterns. For PP cross-stage the agent packs the M per-microbatch payloads into one leading-dim-extended tensor and issues a single dispatch: a masked `xm.all_reduce` over a $(\mathit{half}, M, \mathit{Batch}, \mathit{SeqLen}, D)$ buffer, where *half* is the rank count per pipeline stage (here 112, since the 224 ranks are split into stage 0 and stage 1) and the buffer simultaneously moves the M forward activations through the stage's send/recv pair. The agent recognizes that masked-AR is the only correctness-preserving substitute for `collective_permute` on Neuron, where the latter is broken. The structural cost is one large dispatch instead of M small dispatches, and the wall-clock saving comes from collapsing M Trainium per-dispatch latency floors into one.

TP MLP.. The agent finds the same structural template as for PP cross-stage but applied to the $M \cdot N_{\text{layers}}$ partial-output tensors that the per-microbatch `mark_step` factors out across both microbatches and TP layers. It flattens all of them into a single contiguous 1-D buffer, emits one row-parallel AR, then slices/reshapes back to the $[M][N_{\text{layers}}]$ structure the caller expects. As with PP cross-stage the saving is one large dispatch in place of $M \cdot N_{\text{layers}}$ small ones.

FSDP weight prefetch. The interesting case is FSDP, because the analogous "stack-the-M-shards-and-issue-one-AG" pattern that wins PP cross-stage and TP MLP *does not survive Phase 4's training-shape gate at 224 ranks*: the packed shard tensor it constructs has shape $(M \cdot N_{\text{layers}}, \text{ws}, \text{shard\_size})$ which at the OLMoE Llama-block shapes exceeds the per-rank HBM allocation budget the Neuron compiler tolerates when the gathered intermediate is materialized in a single graph; the attempted compile aborts before the per-step measurement can be taken. This is the kind of hardware-binding constraint that candidate-by-candidate static analysis cannot predict — the simulator's cost model ranks the packed candidate as the best, and the Phase 3 LLM enumerates it confidently — but the search loop catches it at Phase 4 and falls back to the next-best strategy among the candidates Phase 3 produced. That fallback, which is what the deployed FSDP runtime actually is, is the per-microbatch per-layer loop ($M \cdot N_{\text{layers}}$ individual AGs, structurally identical to the developer baseline at the call level). The agent's contribution is therefore *not* dispatch collapsing; it is the choice to lift the per-microbatch `mark_step` that the developer baseline emits between microbatches, letting NeuronCC fuse all $M \cdot N_{\text{layers}}$ AGs into one HLO graph per training step. The inline developer baseline keeps the per-microbatch `mark_step` for the reasons the introductory paragraph of this section enumerates (HBM headroom, NEFF compile latency, in-graph fusion within a microbatch), but at 7-node training scale none of those concerns actually binds, and the agent's single-graph variant collapses M per-graph dispatch and compile-amortization floors into one. The end-to-end ratio (Table 5, FSDP row: 8.52 ms baseline vs 2.01 ms agent at 7-node training) is the agent's contribution even though no per-call collective count changes: the dispatch count is identical, but the graph count drops from M to 1 per step. The broader point this FSDP case makes about the search

loop is the value of Phase 4 hardware validation: the agent's enumerated theoretically-best candidate would have crashed at compile if deployed blindly; the loop catches that and converges instead to the next-best candidate that still wins 4.2× over the developer baseline. The agent's contribution on a problem like FSDP is therefore as much the *robust convergence* to a deployable next-best as it is the discovery of a structurally novel strategy. The non-obvious common thread is that all three problems look like fundamentally different parallelism patterns at the framework level (pipeline, tensor, data) but reduce to the same strategy template once the per-microbatch `mark_step` structure is made explicit; the simulator's four-tier amortization model (Section 3.2) is what lets the agent see that structure without running on hardware.

E Case Study: AG+RS vs Pack-and-Gather

The AllToAllV collective dispatches variable-length per-rank chunks. The strategy AWS practitioners write inside training scripts is "**AG+RS**": pad every rank's data into a buffer of fixed size $N \cdot c_{\max}$ (where $c_{\max}$ is the largest chunk), call one `all_gather` to materialize the global $(N \cdot N \cdot c_{\max})$ buffer, `permute+reshape` to put each destination rank's data together, and call one `reduce_scatter` with sum-reduction and scale $1/N$ to extract each rank's share. Two dispatches, no host-side count computation, clean dataflow. The natural alternative — `xm.all_to_all` — is precluded by a limitation of the underlying Neuron collective-communication library: at multi-node world sizes the compiler rejects the N-element output tuple with an internal instruction-check failure, and even when it does compile (e.g. in a 1-node configuration), it falls back to a slow ring exchange that practitioners universally avoid, so the established substitute is to compose from AG and RS. Practitioners pick AG+RS precisely because in a side-by-side hardware microbenchmark of the kind the AWS Neuron tutorials encourage and demonstrate [2] it wins: 1.66 ms per call vs 2.75 ms for the agent's pack-and-gather strategy [2].

The agent's strategy is different. It packs the variable-length data into a fixed-size send buffer where rank i's payload starts at offset $i \cdot c_{\max}$; calls a single `all_gather` to produce a $(N, N \cdot c_{\max})$ tensor; and on the receive side extracts each rank's slot via `index_select` with a trace-time-precomputed index vector. One dispatch, more local work. A third option the agent considers and rejects is a `collective_permute`-based ring all-to-all: each rank sends its payload to its right neighbor and receives from its left for $N-1$ rounds. The ring is bandwidth-balanced on hardware with asynchronous send/receive (e.g. NCCL on a hierarchical NVLink+IB fabric), but on Trainium each `xm.collective_permute` is a synchronous dispatch with the same per-issue floor as `xm.all_gather`, so the agent's ring proposal pays $N-1$ dispatch floors versus AG+RS's two, and the agent eventually rejects it on the simulator at N=32.

Why AG+RS wins per-call but loses at training scale. The per-call microbenchmark runs 20 iterations with all NEFFs pre-warmed in the Neuron compile cache. In that regime AG+RS's extra dispatch is cheap (the NEFFs are cached) and its *post-collective local work* is just a contiguous `reshape+view` on a permuted buffer — a few microseconds. The agent's `index_select` moves output bytes at random-access bandwidth and looks expensive in isolation.

The training-scale regime is different in three ways. First, every `_A2AV.forward mark_step` pair compiles a fresh HLO graph (the surrounding model graph differs from step to step), and the Neuron runtime under standard large-model training settings keeps only a small window of recently-compiled NEFFs resident on device (see Evaluation Setup, Section 4, for why this cap is set deliberately). The combined AG+RS path produces a graph with two collectives and several intermediate buffers, which produces a larger NEFF and more cache reload events. Second, AG+RS's apparently-cheap `permute+reshape` is contiguity-dependent: when the permutation puts a non-leading dim first, PyTorch silently inserts an O(numel) copy of the entire gathered $(N \cdot N \cdot c_{\max})$ buffer at strided HBM bandwidth, and on Trainium strided bandwidth is roughly 4× slower than sequential (Figure 6). Third, the compilation_cost($b_{\max}$) term scales super-linearly with the largest single-collective tensor, and AG+RS's all-gather materializes a tensor that is $N\times$ the agent's send buffer. None of these costs show up in the 20-iter microbenchmark. All of them show up in end-to-end 250-step training.

What the simulator and the loop catch. The simulator's (4) contiguity-aware implicit-copy term assigns the real cost to AG+RS's `permute+reshape` chain. Its `compilation_cost` term charges AG+RS for the larger graph it induces. Together these flip the predicted ranking: AG+RS is worse at training scale even though it looks better in microbenchmark. Phase 5 ranks by simulator and emits the agent's pack-and-gather strategy. End-to-end at 7-node OLMoE (Section 4), the agent AllToAllV alone produces ≈25% of the 1.40× speedup and Figure 2 summarizes the per-call vs per-step disagreement.

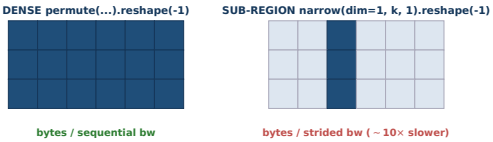

Figure 6. Contiguity-sensitive implicit copies. The same Python `...reshape(-1)` is free on a dense permuted source (left, sequential bw) but pays an $O(\text{numel})$ strided copy on a sub-region narrow source (right, 4×–10× slower).

***Side-by-side code: baseline vs agent AllToAllV.*.** The agent's deployed AllToAllV runtime collapses the baseline's AG+transpose+RS chain (two collectives plus a strided permute/reshape) into a single padded `all_gather` with a metadata-only slice (Figures 7, 8).

```
def baseline_alltoallv(x, send_counts):
    # AG + transpose + RS path (2 collectives)
    cap = max_send_count(send_counts)
    padded = pad_to_cap(x, cap)
    full = xm.all_gather(padded, dim=0)
    full = full.view(ws, ws, cap, D)
    full = full.transpose(0, 1).contiguous()
    y = xm.reduce_scatter(
        xm.REDUCE_SUM, full.view(-1, D),
        scale=1.0, scatter_dim=0,
        shard_count=ws)
    return unpad(y, recv_counts)
```

Figure 7. Internal-AWS-optimized baseline AllToAllV: `all_gather` + strided `permute`/`contiguous` + `reduce_scatter`.

```
def agent_alltoallv(x, send_counts):
    # Pack-and-gather: ONE collective, no strided ops.
    cap = max_send_count(send_counts)
    packed = pack_to_dense(x, cap)
    gathered = xm.all_gather(packed, dim=0)
    gathered = gathered.view(ws, ws, cap, D)
    # Incoming slice is contiguous at outer-axis row `rank`:
    # metadata-only view + slice, no permute, no rs.
    return gathered[:, rank].reshape(-1, D)
```

Figure 8. Agent-emitted AllToAllV (*pack-and-gather*): one collective, dispatch count halved, no strided `permute`/`contiguous`. The metadata-only `view+slice` replaces the chain that costs strided HBM bandwidth in the baseline.

F Cost-model implementation details

This appendix collects the implementation-level details factored out of §3.2 so the main text can stay at the formula-and-idea level.

F.1 Per-op floors and TrackedTensor recording

Pure-metadata view ops — `view`, `narrow`, `transpose`, `permute`, `expand`, `squeeze`, `unsqueeze`, `flatten`, `slice` — pay exactly the per-op floor; charging their isolated-microbench cost (dominated by `mark_step` overhead) would penalize slice-loop chains that XLA actually fuses inside one graph. The `_FUSED_ELEMENTWISE_OPS` category floor-prices `mul`/`add`/`sub`/`div`/`mod`/`neg` because adjacent elementwise ops fuse into one HLO kernel. `TrackedTensor` overrides Python arithmetic dunders to record into the same op counter the named-function calls use; without this an algorithm that interleaves `* inv` between `xm.all_reduce` calls becomes event-stream-indistinguishable from one that does not, and the back-to-back amortization in §F.2 cannot disambiguate the two. The bytes b in every bytes-aware rule come from the actual PyTorch tensor at trace time, not from declared shape.

F.2 Four-tier amortization fitting

The α_i coefficients are fit at Phase 1. `measure_back_to_back_amortization` returns total wall-clock time for N back-to-back `xm.all_reduce` calls in one `mark_step` graph at configurable depth. When the LLM has probed at least depths $\{1, 2, 4, 8\}$, the framework auto-fits each α_i as the marginal per-issue cost in tier i divided by the depth-1 (full) cost. On a 7-node trn1.32xlarge cluster co-located in a single VPC (AWS's per-account network isolation boundary, which the EFA fabric requires because EFA peers must share an L2 broadcast domain), the fit reliably converges to $(\alpha_1, \alpha_2, \alpha_3) \approx (0.30, 0.10, 0.02)$ and would re-fit if the EFA pipeline depth changes. The back-to-back run resets on any non-free-XLA-op (which is why the arithmetic-dunder recording from §F.1 is load-bearing).

F.3 Launch and NEFF-reload constants

The factor 2 in $T_{\text{launch}} = 2\,\ell\,m$ covers the forward `mark_step` and the implicit backward `mark_step` PyTorch/XLA inserts before `loss.backward()` returns. The simulator AST-derives an outer-loop graph-launch tax that contributes to m: each outer `for m in range(M)` / `range(N_MB)` / `range(num_microbatches)` loop (or any outer loop containing an explicit `xm.mark_step()`) charges $2 \times$ `per_graph_neff_load_us` (default 200 μs, doubled for fwd+bwd). Inner per-tensor / per-bucket loops (`for g in rep_grads`, `for bk in buckets`) do not pay this tax: at training scale the candidate function is called once per step and those inner ops fuse into a single back-to-back NEFF chain.

F.4 Structural terms: fusion-credit, HBM safety, primitive-viability

Fusion-credit discount factor is set per cluster by the same Phase-1 calibration that fits α_i; on our cluster the constant is 0.30, fit against bundled-vs-per-microbatch HW measurements across the three Llama-side model-extension primitives in §4.3.

HBM safety uses default budget 2 GB/core, scale 10,000 μs. The AST recognizes hardcoded byte-budget assignments (`bucket_bytes = 32 << 20`, `chunk_bytes`, `partition_bytes`, and their uppercase / `max_*_bytes` / `*_byte_limit` / `*_cap_bytes` variants) and clamps the simulated peak by this cap before evaluating the quadratic penalty. The same cap propagates to the `scaled_bytes` of `cat`/`stack`/`reshape`/`contiguous` ops and to the `scaled_max_bytes` input of $C(b_{\max})$; this prevents a bucketed algorithm from being charged for a ∼30 GB scaled tensor it never actually materializes. The compilation-cost extrapolation $\hat{C}(b)$ also uses the clamped value to avoid log-linear blowup past the largest calibration sample.

Primitive-viability terms encode two real HW failure modes: `xm.collective_permute` ring patterns at world size > 64 trigger a documented Trainium compiler SIGABRT; an `xm.all_reduce` payload past the Neuron Runtime's per-collective tensor-size limit fails at `LoadCollectives`. Both receive $+\infty$ cost.

F.5 Phase-5 gate fallback and rationale

The HW microbench runs 20 iterations of the isolated candidate call under `xla.step()` (this boundary forces a fresh `mark_step` graph per iteration, reporting steady-state cost rather than a once-amortized compile). The training-validation harness runs a 10-step bf16 training pass on an 8-layer DIM=1024 synthetic LM, bf16-only because that is Trainium's native execution precision. When no Phase-3 candidate clears both gates (e.g., every mutation tried to use a cluster-broken primitive at training scale), Phase 5 falls back to the lowest-simulator-score *baseline* template — a human-written seed known to compile and run on this hardware — rather than deploying a sim-only winner that cannot load. This fallback was never triggered on the eight problems evaluated; it is a robustness safety rail.

G Profiling tool details

The five Θ buckets are populated by seven calibration tools the agent invokes through Phase 1's `ToolBox`:

- `measure_algorithm_latency` — parameterized over primitive choice and tensor size for `all_gather`, `reduce_scatter`, `all_reduce`, `collective_permute`, `all_to_all`. Probes world sizes $\in \{4, 32, 224\}$ and payload sizes $\in \{2^k : k = 10, \ldots, 22\}$ bytes, fits $T(b, N) = \alpha(N) + \beta(N)b$ per collective.
- `measure_p2p_transfer` — disambiguates intra-node NeuronLink from inter-node EFA bandwidth.
- `measure_xla_op_overhead` — per-local-op floor.
- `measure_compilation_cost` — fits the per-NEFF reload curve $C(b)$ over the largest collective tensor size (sweep $b \in \{2^{12}, 2^{16}, 2^{20}, 2^{24}\}$).
- `measure_graph_launch_overhead` — ℓ, the per-`mark_step` framework cost, measured as the marginal cost of $k + 1$ no-op collectives in a single graph.
- `measure_memory_copy_throughput` — sequential and strided HBM bandwidth.
- `check_cross_node_support` and `check_primitive_compilation` — guardrails (catch cluster-broken primitives at calibration time, not at deployment).

The LLM picks call sites, sweeps, and aggregation strategy autonomously — it draws on its general knowledge of LLM training, GPU and Trainium collective semantics, and performance-engineering practice — and produces the numerical constants that populate the simulator. We supply the tools (so the LLM cannot fabricate numbers), not the experimental design (so the LLM exploits domain knowledge to choose probes more informative than a hardcoded script would).

H Search-style prompt scaffolds

The three Phase-3 search styles share the Phase-1 simulator and the Phase-4/5 gates but differ in how the LLM-time budget $B = 1 + K + 2R$ calls is spent. The three figures below show the system-prompt shape each style uses (paraphrased to one-page form; the full templates live in `experiments/run_search.py`).

```
[SYSTEM]
You are designing a Trainium collective strategy.
Stage 1 (1 call): emit K=5 structurally distinct
  strategy sketches as (name, description) pairs.
Stage 2 (K=5 calls): implement each sketch as
  runnable Python; sandbox-validated at ws in {4,8}.
Stage 3 (2*R=4 calls): refine top-2 by Sim score.
  Each refine round: identify dominant cost term tau*
  in {T_local, T_coll, T_launch, T_NEFF, T_net};
  emit mutation s' that reduces tau*(s).
[USER]
Problem sig: <signature>
Seed templates: <S_P with Sim scores>
Theta (cost-model params): <flattened Phase-1 table>
```

Figure 9. Strategy-enumerate prompt scaffold (default).

```
C = InitPool(S_P, sigma)
# sigma in {strat-enum, cc-react, multi-island}
for r in range(R):
    s_prime = M.propose(C, Theta)
    if not Verify(s_prime, ref): continue
    score = Sim(s_prime)
    C.add(s_prime, score,
          sigma_specific=True)
```

Figure 14. Phase 3 — LLM-driven search.

```
for s in C.survivors():
    t_hw = Microbench(s, hw, iters=20)
    shape_ok = ShapeGate(s, hw)
    tv_ok = TVGate(s, hw)
    if not (shape_ok and tv_ok):
        s = M.recover(s)  # up to 2 attempts
```

Figure 15. Phase 4 — hardware validation.

```
s_star = min(survived, key=Sim)
emit(f"runtime/trainium_{P.name}.py",
     s_star)
deploy_to(hw)
```

Figure 16. Phase 5 — code generation.

```
[SYSTEM]
You are a Trainium collective-strategy refiner.
Run for 2*R*K = 20 ReAct turns. Each turn:
  Thought: analyze current strategy's bottleneck.
  Action: emit mutation s'.
  Observation: simulator returns Sim(s') and a
    per-term breakdown.
Keep the lowest-Sim strategy.
[USER]
Problem sig: <signature>; seed: <s_0>
Theta: <flattened>
```

Figure 10. CC-ReAct prompt scaffold (single-trajectory).

```
[SYSTEM]
You are a parallel mutation operator. K=5 islands
each hold one strategy. For r in 1..R rounds:
  for each island i:
    propose s'_i = M(s_i)  # LLM call
    if Sim(s'_i) < Sim(s_i): s_i <- s'_i
  every R/2 rounds, swap best of two random islands.
Total: K*R = 20 calls.
[USER]
Seeds: <K samples drawn from S_P>
Theta: <flattened>
```

Figure 11. Multi-island GA prompt scaffold.

I Per-phase code snippets

Each of the five phases corresponds to a compact entry point in the codebase. The five figures below show the call shapes; the full implementations live in `search/run_search.py`.

```
toolbox = build_calibration_tools(
    hw=trainium_7node)
Theta = M.calibrate(toolbox, max_calls=12)
# Theta = (Theta_net, Theta_launch, Theta_NEFF,
#          Theta_copy, Theta_neff_cache)
Sim = BuildCost(Theta)
```

Figure 12. Phase 1 — calibration.

```
pool = []
for s in S_P:
    if Verify(s, ref, ws={4,8}):
        pool.append((s, Sim(s)))
pool.sort(key=lambda x: x[1])  # by Sim
```

Figure 13. Phase 2 — baseline evaluation.